Comparative Analysis of Time Series Forecasting Methods for Rainfall in Jakarta, 2010-2026

Britney Angeline Soeseno
Computer Science & Statistics Department,
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
britney.soeseno@binus.ac.id

Nadya Angelie Lislie
Computer Science & Statistics Department,
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia
nadya.lislie@binus.ac.id

***Abstract* - Flooding in Jakarta has been a recurring problem, causing economic losses of up to IDR 1.69 trillion in 2025, with the worst hitting on January 23, 2026, where 129 flood points were recorded across the city. One way to reduce the impact is by accurately predicting rainfall. In this study, we compare seven rainfall forecasting methods, Seasonal Naive, TSR, ARIMA, SARIMA, Neural Network, LSTM, and GRU, all tested on the same weekly rainfall data from NASA/POWER, spanning January 2010 to January 2026 with 835 total observations. The data was split 80/20, using 668 weeks for training and 167 weeks for testing. Each model is evaluated using RMSE, MAE, and MAPE on the same test set to ensure a fair comparison. Results show that TSR came out on top with a test RMSE of 26.95 mm/week, followed by SARIMA at 29.49 mm/week. Among deep learning models, LSTM performed best with a test RMSE of 32.46 mm/week, followed by GRU at 33.48 mm/week and Neural Network at 33.20 mm/week. All six forecasting models beat both baseline methods, proving that using more structured models is worth it. Overall, for a 12-year dataset with a clear seasonal pattern like Jakarta rainfall, simpler seasonal statistical models can hold their own against, or even beat, more complex deep learning models.**

Keywords—Rainfall Forecasting, Time Series Analysis, SARIMA, Neural Network, LSTM, GRU

I. INTRODUCTION

Jakarta is among Southeast Asia's most flood-vulnerable cities. According to BPBD DKI Jakarta, the highest flood frequencies occurred in January, February, May, June, and December throughout 2020–2025 [1]. The situation worsened significantly on January 23, 2026, when intense rainfall triggered 129 flood points across the city, inundating 55 RT in South Jakarta, 38 RT in West Jakarta, and 34 RT in East Jakarta, with water levels reaching 90 cm in Cipulir [1]. Economic losses from flooding in the greater Jakarta area reached IDR 1.69 trillion in 2025–2026 [2], underscoring the urgent need for reliable forecasting tools to support urban flood management.

Predicting rainfall in a tropical city like Jakarta is not an easy task. The daily rainfall data tends to be messy, inconsistent, and can spike really high during monsoon season. Common statistical models like ARIMA and SARIMA have been widely used for this, but they work based on linear assumptions, which means they might miss some of the more complex and nonlinear patterns that come with monsoon rainfall [3]. On the other hand, deep learning models like LSTM and GRU have come up as newer options that can handle these nonlinear patterns better, but the catch is that they need a lot of training data to actually work well [4].

A key methodological issue in previous studies is inconsistent evaluation scales: some models are fitted on daily data while others use monthly aggregates, making direct comparisons misleading [5]. This study addresses this by aggregating all data and fitting all seven models on the same dataset, enabling a fair, apple-to-apple comparison across statistical and machine learning approaches.

This study contributes three main things: first, we compare seven different forecasting methods using the same rainfall dataset from Jakarta so the comparison is fair. Second, we do a thorough data exploration including stationarity testing and residual diagnostics. Third, we share some practical takeaways on how to pick the right model when you only have a limited amount of historical data to work with.

II. LITERATURE REVIEW

2.1 Baseline Model Seasonal

Seasonal Naive is classical benchmark method widely used as performance baseline in time series forecasting. The Seasonal Naive forecast sets each future period equal to the corresponding period one season earlier, for monthly rainfall, this means predicting October 2024 using October 2023. Makridakis et al. [6] demonstrated in the M5 competition that Seasonal Naive often provides surprisingly competitive forecasts for data with strong seasonal patterns, and that complex models must substantially outperform it to justify their added complexity.

2.2 Time Series Regression with Seasonal Dummies (TSR)

Time Series Regression with seasonal dummy variables is a straightforward approach that captures deterministic seasonal patterns through indicator variables. In rainfall studies, monthly dummy variables effectively encode the monsoon cycle, wet months receive large positive coefficients relative to the reference month, while dry months receive negative ones. Hyndman and Athanasopoulos [7] note that TSR works particularly well when seasonality is stable and predictable, as is the case with Jakarta's annual monsoon. The inclusion of lagged dependent variables in TSR allows it to also capture short-term autocorrelation, improving fit without the formal structure of ARIMA.

2.3 ARIMA

ARIMA, which stands for AutoRegressive Integrated Moving Average, has been one of the most commonly used models in time series analysis for a long time. An ARIMA(p,d,q) model basically works with three components: p for the autoregressive part, d for how many times the data needs to be differenced to become stationary, and q for the moving average part. When it comes to rainfall data specifically, Box and Jenkins [3] suggested that applying a log transformation before fitting the model helps stabilize the variance and makes the distribution look more normal, which leads to better estimation results. Adnan et al. [8]

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

applied ARIMA to monthly precipitation in Pakistan and found it performed well when the series was stationary in mean (ADF test confirmed, d = 0).

2.4 SARIMA

SARIMA is basically an upgraded version of ARIMA that adds seasonal components to better handle data that follows a repeating pattern over time. For monthly data, the seasonal period is set to S = 12, which captures the yearly monsoon cycle. Mohamad et al. [9] found that SARIMA does a better job than regular non-seasonal ARIMA when forecasting monthly rainfall in Malaysia, because it can directly model the seasonal correlations that show up at lags 12 and 24 in the ACF plot. A limitation of SARIMA on short time series is that seasonal parameters require sufficient observations for reliable estimation. With only 57 training months, achieving fully white-noise residuals remains challenging for tropical rainfall data [10].

2.5 Neural Network (NN)

Feedforward neural networks for time series work by taking past values as inputs and running them through one or more hidden layers using nonlinear activation functions like ReLU or sigmoid. This setup, sometimes written as NNAR(p, k), is basically a nonlinear version of autoregression. Zhang [11] showed that neural networks can pick up on nonlinear patterns that ARIMA simply cannot catch, especially in rainfall data where the shift between wet and dry seasons involves more complex, threshold-like behavior. The biggest risk with this model is overfitting, particularly when the training data is small, but techniques like early stopping and dropout regularization can help keep that under control.

2.6 LSTM

Long Short-Term Memory, or LSTM, is a type of recurrent neural network that was introduced by Hochreiter and Schmidhuber [12] to solve the vanishing gradient problem that standard RNNs often struggle with. It works using three gates, which are the forget gate, input gate, and output gate, that control what information gets kept or thrown away at each time step. This makes LSTM really good at capturing patterns that span long periods of time. For rainfall forecasting, Tian et al. [4] found that LSTM beat ARIMA when there was more than ten years of training data available, but the advantage gets smaller with shorter datasets. Since this study uses a relatively limited training window, LSTM may not be able to fully show its strengths compared to simpler models.

2.7 GRU

Gated Recurrent Unit, or GRU, is a simpler version of LSTM that was introduced by Cho et al. [13]. It combines the forget and input gates into one single update gate and removes the separate cell state, which means it has fewer parameters. This makes GRU faster to train and less likely to overfit on smaller datasets. Poornima and Pushpalatha [14] compared LSTM and GRU for daily rainfall prediction and found that GRU trained faster and performed just as well or even better on medium-sized datasets. These qualities make GRU a solid choice for this study where the amount of training data is limited.

III. RESEARCH METHODOLOGY

3.1 Dataset Source

This study utilizes daily meteorological data obtained from the NASA Prediction of Worldwide Energy Resources (POWER) database. The dataset contains weather observations for Jakarta, Indonesia, including precipitation and several supporting atmospheric variables. Among the available parameters, precipitation data were selected as the primary variable of interest for rainfall forecasting analysis.

The original dataset consists of daily observations spanning January 1, 2010 to January 1, 2026, providing approximately 5,845 daily records prior to temporal aggregation. NASA POWER provides satellite-derived and model-assisted climate information with consistent temporal coverage, making it suitable for long-term environmental monitoring and forecasting studies. The rainfall variable used in this study is PRECTOTCORR (Corrected Precipitation), which represents corrected precipitation measurements expressed in millimeters per day (mm/day).

3.2 Data Cleaning

Data cleaning was conducted to ensure that the rainfall dataset was suitable for time series analysis and forecasting. The date variable was first converted into a standardized date format to preserve chronological order and enable accurate temporal aggregation.

The rainfall variable (PRECTOTCORR) was examined for missing and inconsistent observations. During the aggregation process, missing rainfall values were handled using omission within the summation procedure (na.rm = TRUE) to avoid disrupting the continuity of the rainfall series while maintaining the available observations.

Unlike many cross-sectional datasets, rainfall naturally exhibits high variability and occasional extreme precipitation events. Therefore, large rainfall values were not automatically treated as outliers or removed from the dataset. Only observations identified as missing or computationally problematic were addressed, while extreme rainfall events were retained to preserve the climatic characteristics and natural variability of rainfall patterns in Jakarta.

3.3 Descriptive Statistics and Exploratory Data Analysis

Before jumping into the forecasting part, we first did some descriptive statistics and Exploratory Data Analysis (EDA) to get a better understanding of how the rainfall data behaves over time. This started with a basic check of the dataset, like making sure the structure was correct, counting the observations, and verifying that data was available for every year without any gaps.

We then calculated summary statistics for the daily rainfall, including the mean, median, standard deviation, minimum, maximum, quartiles, skewness, and kurtosis, to get a sense of how the data is distributed. Since rainfall data tends to be right-skewed, we also plotted a histogram and compared it side by side with the log(1+x) transformed version to see how the transformation affects the distribution.

To look at how rainfall changes over time, we used time series plots to visualize the daily fluctuations across the whole observation period. We also plotted monthly rainfall totals using trend and seasonal plots to get an early idea of how aggregating the data affects the patterns. On top of that,

monthly boxplots were used to check how rainfall varies across different months of the year.

For extreme rainfall values, we made outlier plots just to visualize them, but we did not automatically remove them since very high rainfall events can still be real and valid data points. Lastly, we applied time series decomposition to the aggregated series to visually break down the trend, seasonal, and irregular components that might affect how well the forecasting models perform.

3.4 Data Aggregation Strategy

The original daily rainfall data were aggregated into two temporal resolutions: monthly and weekly rainfall series. This dual aggregation strategy was implemented to examine how temporal resolution influences forecasting performance and model behavior.

Monthly aggregation was performed by summing daily rainfall values within each calendar month to produce rainfall totals expressed in millimeters per month (mm/month). In addition, weekly aggregation was constructed using consecutive rolling 7-day blocks beginning from the earliest available observation. For each block, rainfall totals were calculated by summing daily precipitation values, while corresponding week-start and week-end dates were recorded. Unlike calendar-based or ISO week definitions, this approach ensures consistent seven-day intervals throughout the observation period and produces rainfall totals expressed in millimeters per week (mm/week).

The monthly series was treated as a preliminary exploratory aggregation, whereas the weekly series served as the primary analytical dataset for forecasting model development.

3.5 Time Series Preprocessing

To prepare the data for modeling, we split it into training and testing sets while keeping the original time order intact. Since time series data is sequential and each observation is connected to the ones before it, randomly shuffling the data would not make sense here. We used an 80/20 split, where the first 668 observations from January 2010 to around October 2022 were used for training, and the remaining 167 observations from around October 2022 to January 2026 were saved for testing to see how well the models perform on unseen data.

Before fitting any model, we also checked whether the rainfall values needed to be transformed using the log(1+x) function. To decide this, we used Box-Cox analysis to look at how stable the variance is and how the data is distributed. We also tested for stationarity using the Augmented Dickey-Fuller (ADF) test, since models like ARIMA assume that the statistical properties of the data stay roughly consistent over time. If the data turned out to be non-stationary or had unstable variance, we would apply log transformation or differencing before moving on to model fitting. The results of these checks are discussed further in the analysis section.

3.6 Model Selection and Justification

This study compares seven forecasting methods representing baseline, statistical, and deep learning approaches. Baseline model is included to establish reference performance and consisted of Seasonal Naive forecasting. This method provide simple benchmark and allow more sophisticated models to be evaluated relative to straightforward forecasting rules.

Statistical forecasting approaches consisted of Time Series Regression (TSR), ARIMA, and SARIMA models. TSR incorporates trend, seasonal, and lag-based information through regression modelling and provides an interpretable representation of rainfall dynamics. ARIMA models capture autoregressive and moving-average structures after considering stationarity conditions, while SARIMA extends ARIMA by explicitly modelling seasonal dependence that commonly appears in rainfall time series.

To go beyond the traditional statistical models, we also tried out some machine learning and deep learning approaches. Specifically, we used feedforward Neural Networks, Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU) models. Neural networks are good at picking up on nonlinear patterns in the data, while LSTM and GRU are built specifically to handle sequential data and remember patterns that stretch over longer periods of time.

3.7 Forecasting Procedure

Model development followed a consistent forecasting workflow to ensure fair comparison across methods. Monthly aggregation was first analyzed as a preliminary modelling exercise to provide initial insight into rainfall dynamics under a smoother temporal resolution. Forecasting models were subsequently applied to the weekly aggregation, which served as the primary forecasting framework due to its larger sample size and richer temporal information.

For ARIMA and SARIMA models, several candidate specifications were estimated and compared using information criteria and residual diagnostics. Model selection considered parameter significance, Akaike Information Criterion (AIC), residual independence, and overall diagnostic adequacy.

Forecasts generated from all models were evaluated on the testing subset using recursive out-of-sample prediction procedures. This approach mimics practical forecasting conditions by assessing predictive performance on previously unseen observations.

Residual diagnostics were also conducted for statistical models to examine whether residuals behaved as approximately random processes. Diagnostic procedures included normality assessment and autocorrelation checks to evaluate model adequacy and identify potential specification problems.

3.8 Model Evaluation

To check how accurate each model is, we calculated three error metrics using the test set: Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE).

RMSE calculates the square root of the average squared errors, so it gives more weight to larger mistakes. MAE looks at the average size of the prediction errors and gives the result in the same unit as the rainfall data, which makes it straightforward to understand. MAPE measures the error as a percentage of the actual values, which is useful for comparing models against each other.

That said, MAPE has a known problem when the actual values are zero or very close to zero. Since Jakarta does have dry weeks where rainfall hits 0 mm, dividing by zero makes

the formula fall apart and produces extremely large or undefined numbers. This is why the MAPE values in this study sometimes go into the thousands of percent, which honestly does not tell us much about whether a model is good or bad. Because of this, even though we still show MAPE in the result tables, we rely mainly on RMSE and MAE as the primary metrics since they stay reliable regardless of whether rainfall is zero or not.

For ARIMA and SARIMA candidate selection, Akaike Information Criterion (AIC) was additionally considered as an in-sample model comparison measure. However, final model performance and ranking were determined primarily using out-of-sample forecasting accuracy obtained from the testing period.

The combined use of these evaluation criteria enables comprehensive assessment of forecasting accuracy while supporting reliable comparison across statistical and deep learning approaches.

IV. DATA ANALYSIS

4.1 Dataset Overview and Descriptive Analysis

The daily rainfall dataset obtained from NASA POWER consists of 5,845 observations covering the period from 1 January 2010 to 1 January 2026. The dataset includes calendar variables (year, month, and day of year), corrected precipitation measurements (PRECTOTCORR), and a standardized date variable used for chronological analysis.

Inspection of the dataset structure confirmed that all variables were successfully imported and converted into appropriate types. Observation counts by year indicate complete temporal coverage throughout the study period. Leap years, namely 2012, 2016, 2020, and 2024, contain 366 observations, while other years contain 365 observations. The final year, 2026, includes only one available observation corresponding to 1 January 2026.

Data quality assessment showed that the rainfall series was complete and internally consistent. No missing values, sentinel values (−999), or duplicate dates were identified. A total of 263 zero-rainfall observations were recorded and retained, as these represent valid dry-day conditions rather than data errors. Overall, the dataset demonstrates high completeness and suitability for subsequent time series analysis.

TABLE I. DESCRIPTIVE STATISTICS OF DAILY RAINFALL

MEASURE	VALUE
N	5845
Mean	6.581
Median	3.940
SD	8.815
Min	0.000
Q1	1.220
Q3	8.380
Max	106.280
Skewness	3.521
Kurtosis	22.769

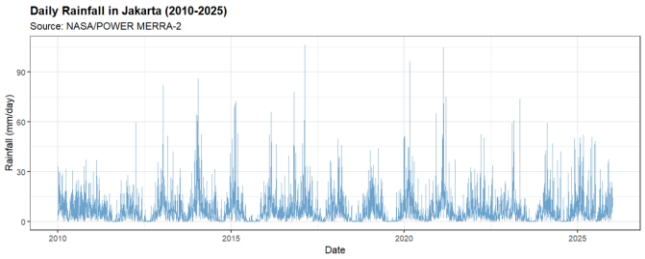
The average daily rainfall was 6.58 mm/day, with a median of 3.94 mm/day, indicating that rainfall distribution is concentrated at relatively low values. Rainfall variability is substantial, as reflected by a standard deviation of 8.82 mm/day and a maximum recorded rainfall of 106.28 mm/day. Quartile values further indicate that most daily rainfall observations remained below 8.38 mm/day, while occasional heavy rainfall events generated considerably larger values.

The distribution exhibits strong positive skewness (3.52) and high kurtosis (22.77), indicating a highly right-skewed and heavy-tailed rainfall pattern. These characteristics are commonly observed in precipitation data, where many days experience little or no rainfall while a smaller number of days experience intense precipitation.

4.2 Exploration Data Analysis

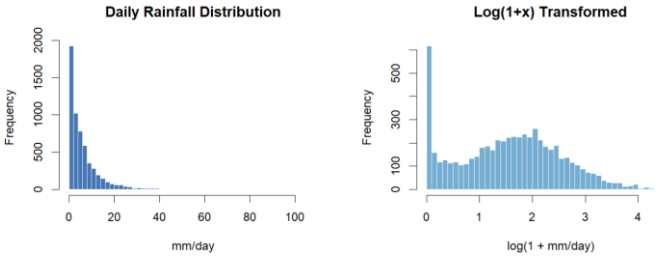
4.2.1 Daily Rainfall Time Series Plot

The daily rainfall plot demonstrates substantial short-term fluctuation with intermittent high-intensity rainfall events.



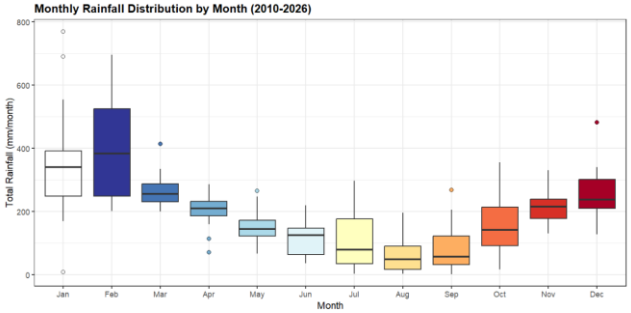
Although rainfall occurs throughout the study period, variability appears uneven over time, with alternating periods of relatively dry and wet conditions. Looking at the raw daily data, there is no clear sign that rainfall is consistently going up or down over time, but we can still see that rainfall tends to cluster around certain seasons.

4.2.2 Rainfall Histogram and Log(1+x) Histogram



The original rainfall distribution is highly concentrated near lower rainfall values with a long right tail, confirming the positive skewness observed in the descriptive statistics. After applying the log(1+x) transformation, the distribution becomes more balanced and less dominated by extreme rainfall values. Although the transformed series does not become perfectly normal, the transformation moderates variance and reduces the influence of unusually large precipitation events, making it more suitable for subsequent forecasting analysis.

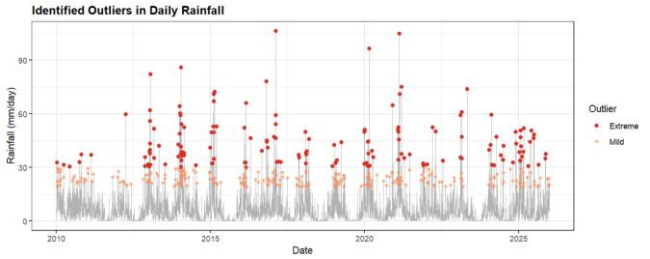
4.2.3 Monthly Rainfall Distribution



The monthly boxplots indicate clear seasonal variation in Jakarta rainfall patterns. Higher rainfall totals are generally observed during the early and late parts of the year, particularly in January, February, November, and December, while lower rainfall totals occur during the middle of the year, especially around July and August. This pattern is consistent with the monsoonal rainfall characteristics commonly observed in Jakarta.

Rainfall variability also differs substantially across months. Wet-season months exhibit wider interquartile ranges and more extreme observations, indicating greater precipitation instability during these periods. In contrast, dry-season months display relatively lower variability and smaller rainfall totals.

4.2.4 Rainfall Outlier Plot



To take a closer look at extreme rainfall values, we used the Interquartile Range (IQR) method to identify outliers. Any observation that goes beyond 1.5 × IQR above the third quartile was labeled as a mild outlier, while anything beyond 3 × IQR was considered an extreme outlier.

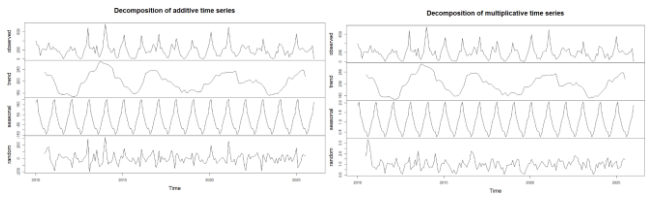
Both mild and extreme rainfall events showed up throughout the observation period, especially during the wetter months. To make sure these unusually large values do not throw off the model estimation too much, we applied winsorization to the extreme values that went above the 3 × IQR upper bound. This way, we kept all the data points but just capped the really extreme ones so they have less influence on the results.

4.3 Monthly Aggregation and Exploratory Analysis

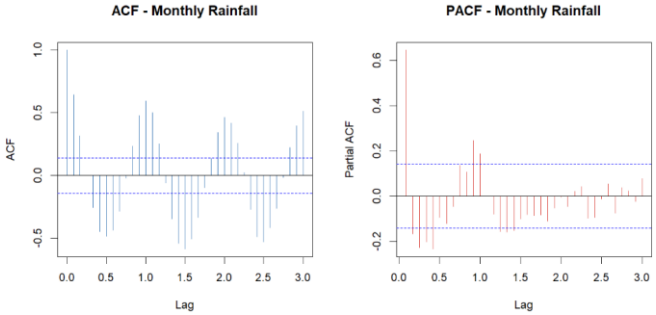
Daily rainfall observations were aggregated into monthly total rainfall to provide a smoother representation of long-term rainfall behavior in Jakarta. The resulting monthly dataset consisted of 193 observations covering the period from January 2010 to January 2026.

Descriptive statistics indicate substantial variability in monthly rainfall totals. The average monthly rainfall was 199.30 mm/month with a standard deviation of 137.18 mm/month. Monthly rainfall ranged from 2.16 mm/month to 768.95 mm/month, indicating the presence of both extremely dry and extremely wet periods. The positive skewness value (1.224) suggests that the rainfall distribution is moderately right-skewed, where several months experienced unusually high precipitation.

The monthly rainfall time series plot shows a recurring annual rainfall cycle with noticeable fluctuations across years. Several periods exhibit sharp rainfall peaks exceeding 600 mm/month, particularly during the wet season. At the same time, prolonged low-rainfall periods can be observed during the dry season, indicating clear seasonal behavior in Jakarta's rainfall pattern.



The decomposition analysis further confirms the existence of a stable seasonal component. Both additive and multiplicative decomposition structures show repeating annual rainfall cycles across the observation period. The trend component also indicates gradual long-term fluctuations in rainfall intensity over time. These findings support the use of seasonal forecasting approaches such as SARIMA and seasonal regression models.



Looking at the ACF and PACF plots, we can see that there is a clear relationship between rainfall values at different points in time, especially around the seasonal lags. This basically means that the rainfall at any given point is influenced by what happened in the previous weeks or months, which gives us a good reason to include autoregressive and seasonal components when building the forecasting models.

4.3.1 Variance Stability and Transformation Assessment

Variance stability of the monthly training series was examined using the Box–Cox power transformation procedure. Rather than conducting a formal hypothesis test, the estimated transformation parameter $\hat{\lambda}$ and its 95% confidence interval were used to assess whether a transformation is warranted. The Box–Cox family is defined as:

$$Y(\lambda) = \begin{cases} \dfrac{Y^{\lambda} - 1}{\lambda}, & \lambda \neq 0 \\ \ln(Y), & \lambda = 0 \end{cases}$$

where $\lambda = 1$ corresponds to no transformation and $\lambda = 0$ corresponds to a log transformation. The estimated parameter for the monthly series was:

$$\hat{\lambda} = 0.9347$$

95% CI: [0.52, 1.35]

Since the value $\lambda = 1$ is contained within the 95% confidence interval, there is no statistical evidence that a transformation is necessary. The variance of the monthly rainfall series is considered reasonably stable, and the original (untransformed) scale is appropriate for modeling. For internal consistency with the log1p preprocessing pipeline adopted in the neural network models, however, the log(1+x) transformation was applied to the statistical models as well. Given that $\hat{\lambda}$ is close to 1, this transformation has a negligible effect on variance structure and does not alter the analytical conclusions.

4.3.2 Stationarity Testing and Differencing Assessment

Stationarity was evaluated using the Augmented Dickey–Fuller (ADF) test.

H_0:The time series is non-stationary

H_1:The time series is stationary

The significance level used was:

$$\alpha = 0.05$$

The estimated Box–Cox parameter obtained from the monthly rainfall series was:

Dickey–Fuller statistic = -7.9585

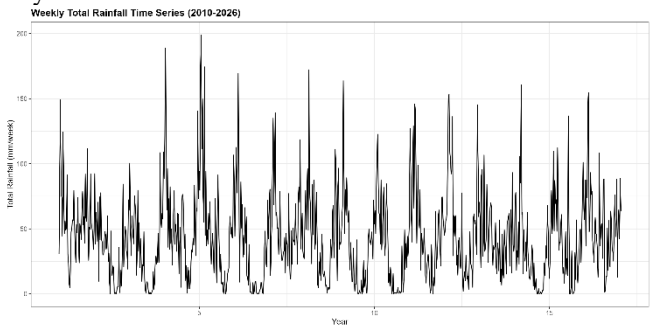
p-value = 0.01

Because the p-value was smaller than 0.05, $H_0$was rejected. The monthly rainfall series satisfies the stationarity assumption and does not require additional non-seasonal differencing ($d = 0$). Consequently, ARIMA and SARIMA models could be estimated directly on the transformed monthly rainfall series.

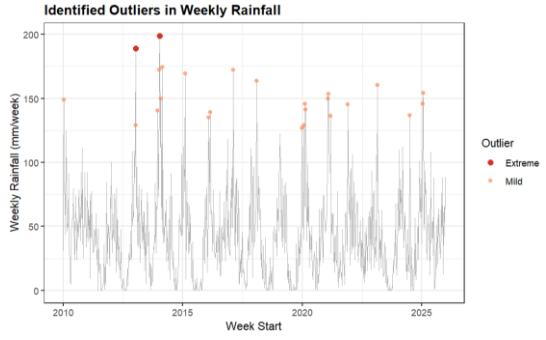
4.4 Weekly Aggregation and Exploratory Analysis

Daily rainfall observations were aggregated into weekly total rainfall to capture shorter-term fluctuations and higher-frequency dynamics in Jakarta's rainfall pattern. The resulting weekly dataset consisted of 835 observations, covering the period from January 2010 to January 2026.

Descriptive statistics indicate substantial variability at the weekly level. The average weekly rainfall was 43.36 mm/week with a standard deviation of 34.02 mm/week. Weekly rainfall ranged from 0 mm/week to 198.94 mm/week, reflecting the presence of completely dry weeks as well as extreme rainfall events. The skewness value of 1.168 indicates a moderately right-skewed distribution, suggesting that extreme rainfall weeks occur more frequently relative to a symmetric distribution.

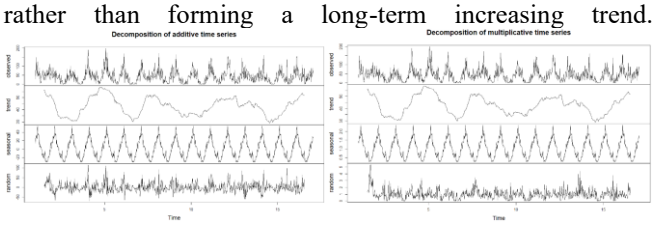


The weekly time series plot shows more irregular and volatile fluctuations compared to the monthly series. While seasonal patterns remain present, they are less visually smooth due to higher noise at the weekly frequency. Sharp spikes in rainfall can be observed, indicating short-duration extreme weather events that are not fully visible in aggregated monthly data.

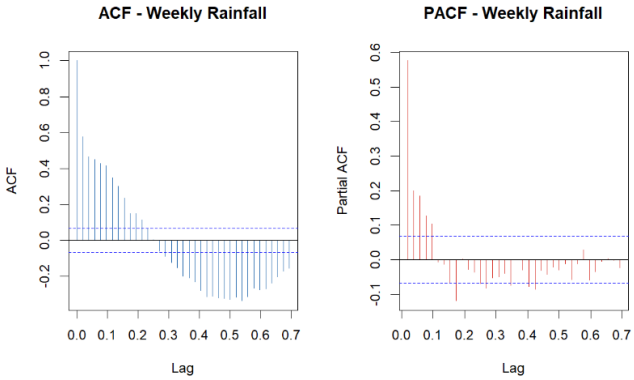


Most weeks have normal rainfall, generally below 120 mm/week. There are a number of mild outliers (orange points), indicating weeks with unusually high rainfall compared to the typical weekly pattern. Only two extreme outliers (red points) were detected, both around 2013–2014, with weekly rainfall close to 190–200 mm/week.

After 2014, there are still several high-rainfall weeks (orange points), but none exceeded the extreme-outlier threshold. The outliers are sporadic rather than clustered, suggesting that extreme rainfall events occur occasionally rather than forming a long-term increasing trend.



The decomposition results indicate that while both additive and multiplicative structures can be applied, the additive form provides a clearer representation of the seasonal pattern. This is evidenced by the varying amplitude of the seasonal component, where periods of higher rainfall exhibit stronger seasonal fluctuations compared to low rainfall periods. Therefore, the additive decomposition is considered more appropriate for representing the weekly rainfall dynamics.



Autocorrelation analysis using the ACF and PACF plots reveals strong short-term dependence, particularly at early lags. The ACF decays gradually, while the PACF shows significant spikes at low lags, suggesting the presence of autoregressive structure. Seasonal patterns are less sharply defined than in the monthly case but remain detectable, supporting the inclusion of both non-seasonal AR terms and seasonal components in model specification.

4.4.1 Variance Stability and Transformation Assessment

The same Box–Cox procedure was applied to the weekly training series. The estimated transformation parameter and its 95% confidence interval were:

$$\hat{\lambda} = 0.9333$$

95% CI: [0.64, 1.22]

The value $\lambda = 1$ falls within the 95% confidence interval, indicating that no transformation is statistically required. The weekly rainfall variance is sufficiently stable for modeling on the original scale. As with the monthly series, the log(1+x) transformation was applied for pipeline consistency with the deep learning preprocessing, and the proximity of $\hat{\lambda}$ to 1 ensures this choice does not materially affect variance properties or model estimates.

4.4.2 Stationarity Testing and Differencing Assessment

Stationarity was evaluated using the Augmented Dickey–Fuller (ADF) test.

H_0:The time series is non-stationary

H_1:The time series is stationary

The significance level used was:

$$\alpha = 0.05$$

The estimated Box–Cox parameter obtained from the weekly rainfall series was:

Dickey–Fuller statistic = -6.685

p-value = 0.01

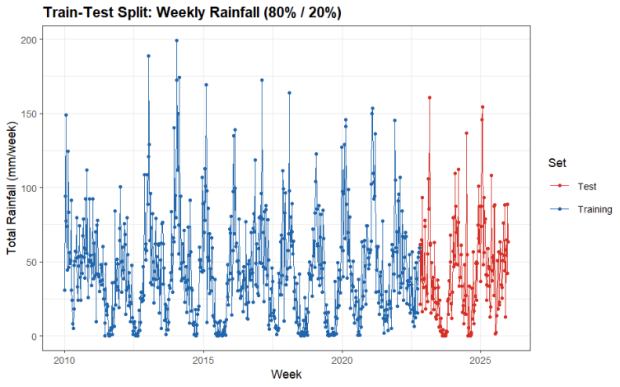
Because the p-value was smaller than 0.05, H_0 was rejected. The weekly rainfall series satisfies the stationarity assumption and does not require additional non-seasonal differencing ($d = 0$). Consequently, ARIMA and SARIMA models could be estimated directly on the transformed weekly rainfall series.

4.5 Train–Test Split

To evaluate model performance, the weekly rainfall dataset was divided into training and testing subsets using an 80/20 split. The partition was applied sequentially rather than randomly to preserve the temporal ordering of the data, which is essential in time series analysis.

The dataset consisted of 835 weekly observations. Based on this, the split resulted in:

- Training set: 668 observations (January 2010 – October 2022)
- Test set: 167 observations (October 2022 – January 2026)



The training set was used to estimate model parameters, while the test set was reserved for out-of-sample evaluation to assess forecasting accuracy. The training period (blue) captures the long-term rainfall behavior, including seasonal patterns and variability, while the test period (red) represents unseen data used to assess forecasting performance. The transition between the two periods appears smooth, with no visible structural break, indicating that the split is appropriate for out-of-sample evaluation.

Consistent with the earlier exploratory analysis, the weekly series remains highly volatile in both subsets, with frequent spikes and short-term fluctuations. These characteristics highlight the challenge of modeling rainfall at a weekly frequency.

In the subsequent analysis, all preprocessing and modeling steps are conducted using the weekly aggregated data. The monthly aggregation is not used in the modeling process at this stage, but will be introduced later as a basis for comparison to evaluate the effect of aggregation level on model performance.

4.6 Baseline Model: Seasonal Naïve

The Seasonal Naïve model serves as the simplest possible forecasting benchmark. It assumes that each future observation is equal to the value observed at the same seasonal position in the most recent cycle. For weekly rainfall with a seasonal period of s = 52, each forecast is defined as:

$$\hat{Y}_{(t+1)} = Y_{(t+1-s)} = Y_{(t+1-52)}$$

where $Y_{(t+1-52)}$ is the observed weekly rainfall value from exactly 52 weeks prior. This model incorporates no trend, no autoregressive structure, and no error-correction mechanism. It is purely replicative, making it useful as a lower-bound reference point. If more sophisticated models cannot meaningfully outperform the Seasonal Naïve forecast, their added complexity is not justified.

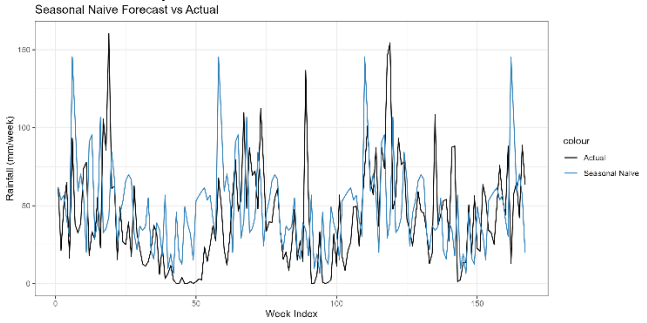
The recursive out-of-sample forecasting procedure was applied over the 167-week test period, with each forecast drawing on corresponding value from previous annual cycle.

TABEL II. SEASONAL NAÏVE FORECASTING PERFORMANCE

Set	RMSE	MAE	MAPE (%)
Train	36.0425	26.9668	657.37
Test	39.3476	30.2422	2725.9

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.

The Seasonal Naïve model achieves a test RMSE of 39.34 mm/week and a test MAE of 30.24 mm/week. The extremely high MAPE values (657% train, 2726% test) are a consequence of near-zero or zero rainfall weeks in the denominator; MAPE becomes numerically unstable when observed values approach zero. Consequently, RMSE and MAE are treated as the primary evaluation criteria throughout this study. These results establish the performance floor that all subsequent models must surpass. Here is the visualization of the model's performance:



4.7 Time Series Regression (TSR)

The Time Series Regression approach models weekly rainfall as a linear function of trend, seasonal dummy variables, and autoregressive lag terms. Five candidate specifications of increasing complexity were evaluated. The general form of the estimated model is:

$$Y(t) = \beta_0 + \beta_1 + \sum_{k=2}^{52} \gamma_k \cdot W_k(t) + \phi_1 Y_{(t-1)} + \phi_2 Y_{(t-2)} + \phi_3 Y_{(t-3)} + \varepsilon_t$$

where $Y(t)$ is weekly rainfall at time t and t is a linear time index capturing long-term trend. $W_k(t)$ are binary seasonal dummy variables for each of the 52 calendar weeks, with Week 1 retained as the reference category (so that 51 dummies enter the model, k = 2, …, 52). $Y_{(t-1)}$, $Y_{(t-2)}$, and

$Y_{(t-3)}$ are the first three autoregressive lag terms. β_0 is the intercept. β_1 is the trend slope. γ_k are week-specific seasonal coefficients. ϕ_1, ϕ_2, ϕ_3 are autoregressive coefficients and ε_t is the error term. Then, Five candidate models were estimated, progressively adding lag terms to examine their marginal contribution:

TABEL III. TSR MODEL ERROR COMPARISON (TRAINING SET)

Model	Specification	RMSE	MAE	MAPE
M1	Seasonal Dummies Only	25.7931	19.5655	859.89
M2	Trend + Dummies	25.7926	19.5616	859.31
M3	Lag1 + Trend + Dummies	24.1241	18.1488	656.71
M4	Lag1 + Lag2 + Trend + Dummies	24.0649	18.0230	629.07
M5	Lag1 + Lag2 + Lag3 + Trend + Dummies	23.8055	17.8261	568.40

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.

TABEL IV. TSR MODEL MORE COMPARISON (TRAINING SET)

Model	Specification	Adj. R^2	AIC	DW
M1	Seasonal Dummies Only	0.3828	6315.83	1.2872
M2	Trend + Dummies	0.3818	6317.80	1.2873
M3	Lag1 + Trend + Dummies	0.4583	6230.86	2.0381
M4	Lag1 + Lag2 + Trend + Dummies	0.4601	6229.59	2.0091
M5	Lag1 + Lag2 + Lag3 + Trend + Dummies	0.4708	6217.18	2.0178

Model M5, incorporating three lag terms alongside trend and seasonal dummies, achieved the lowest training RMSE (23.81), lowest AIC (6217.18), and highest adjusted R^2 (0.4708). The Durbin–Watson statistic of approximately 2.02 indicates the absence of first-order autocorrelation in the residuals, confirming that the lag structure has adequately captured serial dependence. Adding the linear trend term alone (M2 versus M1) produced negligible improvement, suggesting that the seasonal dummy structure already accounts for most explainable variation at the weekly level.

TABEL V. VIF DIAGNOSTICS FOR M5 TSR BEST MODEL

Predictor	GVIF	Df	GVIF^(1/(2·Df))
lag1	2.02	1	1.42
lag2	2.24	1	1.50
lag3	2.02	1	1.42
t	1.00	1	1.00
WEEK_F	2.78	51	1.01

We ended up choosing Model M5 as our final TSR model. To make sure there was no serious multicollinearity issue among the predictors, we checked the Variance Inflation Factors (VIF) for each variable. All VIF values came out below 3.0, which means the level of collinearity between predictors is still within an acceptable range. The WEEK_F factor (seasonal dummies) exhibited a Generalized VIF (GVIF) of 2.78 across 51 degrees of freedom, which on the adjusted scale of GVIF^(1/2Df) = 1.01 is well within acceptable bounds.

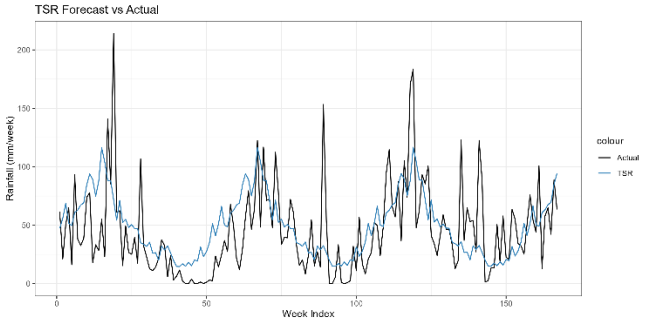
Out-of-sample forecasting for TSR was performed using a recursive procedure. At each step, the three most recent predicted values (or actual training values at the start of the horizon) were used as lag inputs, and the resulting prediction was rolled forward. This ensures the forecast correctly mirrors a real-world scenario in which future observations are unavailable.

TABEL VI. TSR (M5) FORECASTING PERFORMANCE

Set	RMSE	MAE	MAPE
Train	23.8055	17.8261	568.40
Test	26.9513	20.3781	1890.25

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.

The TSR model achieves a test RMSE of 26.95 mm/week, representing a meaningful improvement over both baseline models. The train-to-test RMSE increase (23.81 to 26.95) is moderate, suggesting that the model generalizes reasonably without severe overfitting. The combined use of lag terms and seasonal dummies enables the model to capture both short-term serial dependence and the annual rainfall cycle in Jakarta. Here is the visualization of the model's performance:



4.8 ARIMA and SARIMA Model

4.8.1 Variance Stability and Transformation Assessment

All the preprocessing steps for ARIMA were only done on the training data to make sure there was no data leakage from the test set. As we found in Section 4.4.1, the Box-Cox analysis on the training series gave a lambda value of 0.4450 with a 95% confidence interval of [0.34, 0.55], which does not include 1, meaning that a transformation is actually needed here. So we applied the log(1+x) transformation to help stabilize the variance before fitting the ARIMA model. After getting the forecasts, we then converted everything back to the original scale using the inverse expm1() function.

4.8.2 Stationarity Testing and Differencing Assessment

Stationarity was evaluated using the Augmented Dickey–Fuller (ADF) test.

H_0:The time series is non-stationary

H_1:The time series is stationary

The significance level used was:

$$\alpha = 0.05$$

The estimated Box–Cox parameter obtained from the weekly rainfall series was:

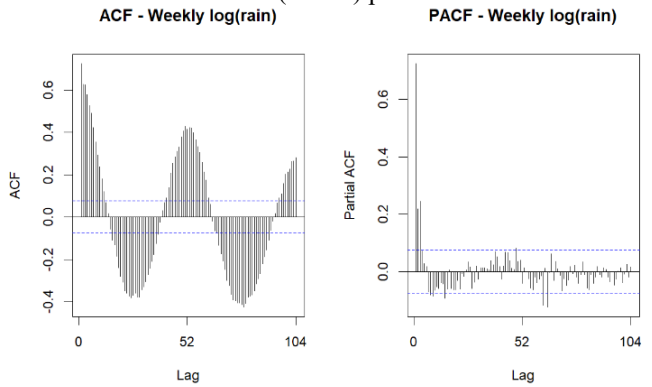
Dickey–Fuller statistic = -6.685

p-value = 0.01

Because the p-value was smaller than 0.05, H_0 was rejected. The weekly rainfall series satisfies the stationarity assumption and does not require additional non-seasonal differencing ($d = 0$). Consequently, ARIMA and SARIMA models could be estimated directly on the transformed weekly rainfall series.

4.8.3 ACF and PACF Identification

Following these preliminary assessments, the identification of candidate ARIMA orders was conducted using the Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) plots.



For ARIMA, we looked at the ACF and PACF plots of the log-transformed training series up to lag 104, which covers two full seasonal cycles. The ACF slowly fades out without any sudden drop, while the PACF shows clear spikes at lags 1, 2, and 3 before dying down toward zero. This kind of pattern usually points to a low-order autoregressive process. There are also some seasonal spikes visible at multiples of lag 52, but they are not as strong as the non-seasonal ones, which suggests that the non-seasonal AR and MA terms should be the main focus. Based on this, we tested candidate orders of $p \in \{1, 2, 3\}$ and $q \in \{0, 1, 2\}$, all with $d = 0$.

For SARIMA, we went back to the ACF and PACF plots but this time paid more attention to the seasonal lags. At lag 52 and its multiples, the ACF shows a noticeable negative spike that gradually fades over the following seasonal lags, while the PACF cuts off sharply after the first seasonal lag. This combination of a slowly decaying seasonal ACF and a sharp cut-off in the seasonal PACF points to a seasonal moving-average term of order $Q = 1$ and a seasonal difference of $D = 1$. For the non-seasonal part, we used the same structure as ARIMA, testing $p \in \{1, 2\}$ and $q \in \{1, 2\}$ as the main candidates.

4.8.4 Model Estimation and AIC Comparison (ARIMA)

Eight ARIMA candidate specifications were estimated on the log-transformed weekly training series, varying the autoregressive (p) and moving-average (q) orders with $d = 0$:

TABEL VII. *ARIMA CANDIDATE AIC COMPARISON*

Rank	Model	AIC
1	ARIMA(3,0,2)	1519.267
2	ARIMA(3,0,1)	1543.041
3	ARIMA(3,0,0)	1544.887
4	ARIMA(2,0,2)	1548.576
5	ARIMA(2,0,1)	1553.079
6	ARIMA(1,0,1)	1553.401
7	ARIMA(2,0,0)	1584.550
8	ARIMA(1,0,0)	1615.230

After that, Coeficient Significance Test was performed to ARIMA model with the lowest AIC:

TABEL VIII. *ARIMA(3,0,2) COEFFICIENT TEST RESULTS*

Parameter	Estimate	Std. Error	z-value	p-value
ar1	2.1732	0.0681	31.90	< 0.001
ar2	−1.4015	0.1275	−10.99	< 0.001
ar3	0.2174	0.0606	3.59	< 0.001
ma1	−1.7474	0.0508	−34.40	< 0.001
ma2	0.7783	0.0476	16.34	< 0.001
intercept	3.3552	0.0827	40.59	< 0.001

All six parameters are significant at the 0.1% level. The model is retained in its full specification.

Then, Residual Diagnostic Testing were performed:

a. Normality Residual Test
   H_0: Residuals are normally distributed
   H_1: Residuals are not normally distributed

   $D = 0.0642$ p-value $= 7.13 \times 10^{-7}$

   Since p-value < 0.05, H_0 is rejected. Residuals do not follow a normal distribution. This is an expected outcome for precipitation data (the bounded, zero-inflated, and right-skewed nature of rainfall propagates non-Gaussian characteristics into residuals even after log transformation.) Non-normality affects prediction interval validity but has no bearing on the point forecast estimates themselves.

b. White Noise Residual Test
   H_0: Residuals are white noise (no remaining autocorrelation)
   H_1: Residual autocorrelation exists

   Q*(lag 12) = 9.92 p-value = 0.6228
   Q*(lag 104, model df = 5) = 101.03 p-value = 0.4243

   Both p-values substantially exceed 0.05. H_0 is failed to be rejected at both lag horizons. Residuals are consistent with white noise, confirming that ARIMA(3,0,2) has fully captured the systematic autocorrelation in the training series. The independence assumption is met.

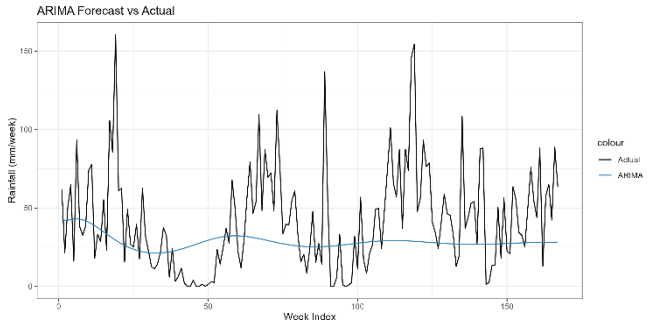
Forecasting Performance of chosen best ARIMA model is as follows:

TABEL IX. ARIMA(3,0,2) FORECASTING PERFORMANCE

Set	RMSE	MAE	MAPE (%)
Train	26.0724	18.4905	254.99
Test	34.4754	25.4518	2802.74

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.

ARIMA(3,0,2) achieves a test RMSE of 34.48 mm/week, ranking sixth among all eight models. The train–test gap (26.07 → 34.48) is larger than for TSR, reflecting the limitation of non-seasonal ARIMA for forecasting over a 167-week horizon where the annual seasonal signal accumulates error. The SARIMA extension directly addresses this by incorporating explicit seasonal components. Here is the visualization of the model's performance:



4.8.5 Model Estimation and AIC Comparison (SARIMA)

Nine SARIMA candidate specifications were estimated on the log-transformed weekly training series:

TABEL X. SARIMA CANDIDATE AIC COMPARISON

Rank	Model	AIC
1	SARIMA(2,0,2)(0,1,1)[52]	1509.884
2	SARIMA(1,0,1)(0,1,1)[52]	1520.941
3	SARIMA(1,0,1)(1,1,1)[52]	1522.940
4	SARIMA(2,0,2)(1,1,0)[52]	1669.199
5	SARIMA(4,0,4)(1,1,0)[52]	1670.289
6	SARIMA(2,0,1)(1,1,0)[52]	1675.516
7	SARIMA(1,0,1)(1,1,0)[52]	1678.801
8	SARIMA(2,0,0)(1,1,0)[52]	1719.433
9	SARIMA(1,0,0)(1,1,0)[52]	1733.688

SARIMA(2,0,2)(0,1,1)[52] achieves the lowest AIC of 1509.884. The models with seasonal MA (0,1,1) structures uniformly outperform those with seasonal AR (1,1,0), confirming that the seasonal moving-average representation is more appropriate for this series. After that, Coeficient Significance Test was performed to the SARIMA model:

TABEL XI. SARIMA(2,0,2)(0,1,1)[52] MODEL COEFFICIENT RESULTS

Parameter	Estimate	Std. Error	z-value	p-value
ar1	0.5182	0.1629	3.18	0.0015
ar2	0.3791	0.1454	2.61	0.0092
ma1	−0.1223	0.1538	−0.80	0.4264
ma2	−0.3981	0.0881	−4.52	< 0.001
sma1	−1.0000	0.1081	−9.25	< 0.001

The ma1 parameter is individually non-significant (p = 0.426). To determine whether this warrants model simplification, the restricted SARIMA(2,0,1)(0,1,1)[52], which removes ma1. These was estimated and compared by AIC:

TABEL XII. FULL VS RESTRICTED SARIMA AIC COMPARISON

Model	AIC	ΔAIC
SARIMA(2,0,2)(0,1,1)[52] full	1509.884	—
SARIMA(2,0,1)(0,1,1)[52] restricted	1516.073	+6.19

The full model achieves a lower AIC by 6.19 units. In model selection practice, a ΔAIC ≥ 2 generally favours the lower-AIC model. A difference of 6.19 is considered substantial. Despite ma1's individual non-significance in the z-test, it contributes meaningfully to the joint model likelihood. Removing ma1 while retaining ma2 degrades the overall moving-average approximation even when the single coefficient appears weak in isolation. The full model is therefore retained.

Then, Residual Diagnostic Testing were performed:

a. Normality Residual Test

H_0: Residuals are normally distributed

H_1: Residuals are not normally distributed

$D = 0.0847$ $\quad$ p-value $= 2.45 \times 10^{-12}$

Since p-value < 0.05, H_0 is rejected. Residuals do not follow a normal distribution. This is the same expected outcome as for ARIMA, attributable to the inherently non-Gaussian characteristics of tropical precipitation. But, this does not invalidate point forecast performance so, the model retains.

b. White Noise Residual Test

H_0: Residuals are white noise (no remaining autocorrelation)

H_1: Residual autocorrelation exists

Q^*(lag 52) = 34.54 $\quad$ p-value = 0.9704

Q^*(lag 104, model df = 5) = 92.29 $\quad$ p-value = 0.6702

Both p-values substantially exceed 0.05. H_0 is failed to be rejected at both lag horizons. Residuals are consistent with white noise. The independence assumption is met.

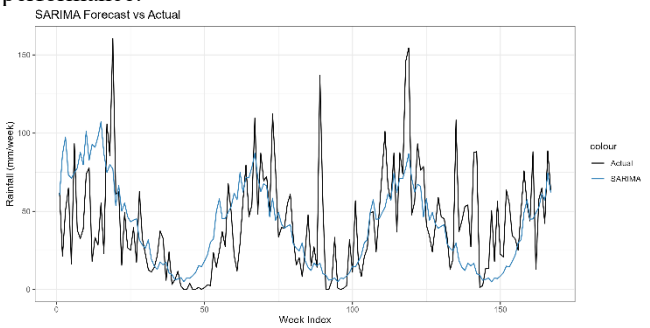
Forecasting Performance of chosen best SARIMA model is as follows:

TABEL XIII. SARIMA(2,0,2)(0,1,1) FORECASTING PERFORMANCE

Set	RMSE	MAE	MAPE (%)
Train	24.4486	16.6414	217.43
Test	29.4886	20.8390	908.05

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.

SARIMA achieves a test RMSE of 29.49 mm/week and MAE of 20.84 mm/week, ranking second overall. The improvement over non-seasonal ARIMA (ΔRMSE = −4.99 mm/week) directly reflects the contribution of the seasonal difference (D = 1) and the sma1 term in capturing Jakarta's annual rainfall cycle. Here is the visualization of the model's performance:



4.9 Neural Network (FeedForward NN)

We built a feedforward Neural Network (NN) using PyTorch. The model takes in the 52 most recent weekly rainfall values as input, which covers exactly one full year of seasonal data, then passes them through one or two fully connected hidden layers using ReLU activation and Dropout regularization with a rate of 0.2, and finally outputs a single forecast for the next week. All input values were scaled to a range of [0, 1] using a MinMaxScaler that was fitted only on the training data to avoid any data leakage. We also made sure

to use the same rolling 7-day block weekly aggregation that was used in the R models, so all 835 observations are exactly the same across every model we tested.

For training, we used the Adam optimizer with a learning rate of 0.001, a batch size of 16, and a maximum of 300 epochs. We also applied early stopping with a patience of 30, based on a held-out internal validation set that takes up 10% of the trailing portion of the training data. In total, six different architectural configurations were tested:

TABEL XIV. NEURAL NETWORK ARCHITECTURE COMPARISON

Config	Structure	Test RMSE	Test MAE	Test MAPE (%)	Test SMAPE (%)
NN(52, 1HL-16)	[16]	33.731	23.233	2188.4	65.07
NN(52, 1HL-32)	**[32]**	**33.196**	**22.646**	**1614.4**	**63.88**
NN(52, 1HL-64)	[64]	33.686	22.972	1929.9	62.79
NN(52, 1HL-128)	[128]	34.152	23.988	1544.5	65.33
NN(52, 2HL-64,32)	[64, 32]	34.945	24.335	2258.9	64.00
NN(52, 2HL-128,64)	[128, 64]	35.369	24.618	1614.6	65.74

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.

The best-performing configuration is NN(52, 1HL-32) with a single hidden layer of 32 neurons, achieving a test RMSE of 33.20 mm/week. Deeper and wider configurations did not improve performance: the two-layer models were outperformed by shallower ones, consistent with the view that 668 training observations spanning 13 annual cycles are insufficient to benefit from increased model complexity. All configurations triggered early stopping well before the maximum epoch limit.

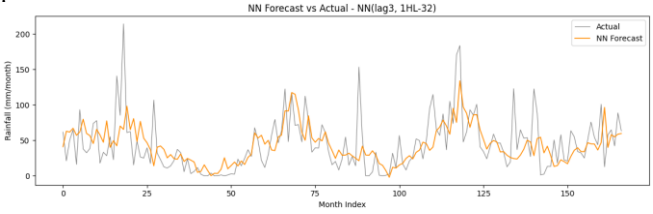
Forecasting Performance of chosen best NN model is as follows:

TABEL XV. BEST NN (1HL-32) FORECASTING PERFORMANCE

Set	RMSE	MAE	MAPE (%)
Train	28.0248	18.2106	414.25
Test	33.1963	22.6467	1614.42

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.

The best NN achieves a test RMSE of 33.20 mm/week, ranking fourth overall. The moderate train-to-test RMSE increase (28.02 to 33.20) indicates that the shallow architecture generalizes better than wider or deeper alternatives. Here is the visualization of the model's performance:



4.10 Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) is a type of recurrent neural network that is built to capture long-range patterns in sequential data through gated memory cells. Each LSTM unit has three gates, which are the input gate, forget gate, and output gate, that control how information flows from one time step to the next. This allows the model to decide what to remember and what to forget over long sequences, which makes it a good fit for weekly rainfall data where seasonal patterns can stretch across 52 steps or more.

For our setup, the LSTM took in input sequences of length 52. The final hidden state was then passed through a Dropout layer with a rate of 0.2 and a linear output layer to produce the forecast. We trained the model using the Adam optimizer with a learning rate of 0.001, a batch size of 16, and a maximum of 200 epochs. Early stopping with a patience of 25 was also applied to prevent overfitting. Six different configurations were tested:

TABEL XVI. LSTM ARCHITECTURE COMPARISON

Config	Structure	Test RMSE	Test MAE	Test MAPE (%)	Test SMAPE (%)
LSTM (16, 1L)	16	1	32.503	23.298	1915.80
LSTM (32, 1L)	32	1	32.589	23.363	1992.03
LSTM (64, 1L)	**64**	**1**	**32.457**	**23.106**	**1803.62**
LSTM (128, 1L)	128	1	32.577	23.499	2084.05
LSTM (32, 2L)	32	2	32.536	23.376	1862.12
LSTM (64, 2L)	64	2	32.539	23.429	2033.44

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.

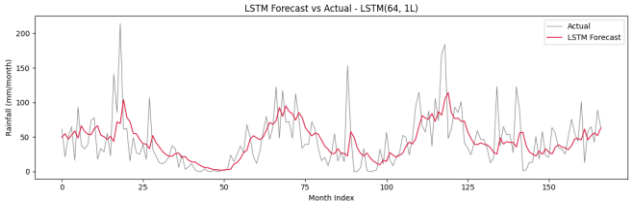
The best LSTM is the single-layer model with 64 hidden units (test RMSE = 32.46 mm/week). All six configurations fall within a narrow 0.14 mm/week RMSE band, indicating that LSTM performance on this dataset is largely insensitive to architectural variation within the evaluated range. Forecasting Performance of chosen best LSTM model is as follows:

TABEL XVII. BEST LSTM (64, 1L) FORECASTING PERFORMANCE

Set	RMSE	MAE	MAPE (%)
Train	32.6972	21.1265	388.89
Test	32.4571	23.1068	1803.62

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.

LSTM achieves a test RMSE of 32.46 mm/week, ranking third overall. The near-identical train and test RMSE (32.70 compared to 32.46) is notable. Test performance marginally exceeds training, likely because the internal validation split for early stopping was comparatively more demanding than the broader test set. Here is the visualization of the model's performance:



4.11 Gated Recurrent Unit (GRU)

Gated Recurrent Unit (GRU) is a simpler version of LSTM that combines the forget and input gates into one single update gate and uses a reset gate instead, which cuts down the number of parameters while still being able to capture long-range patterns in the data. For the training setup, we used exactly the same configuration as the LSTM, which includes an input sequence length of 52, the Adam optimizer with a learning rate of 0.001, a batch size of 16, a maximum of 200 epochs, early stopping with a patience of 25, and MinMaxScaler normalization.

TABEL XVIII. GRU ARCHITECTURE COMPARISON

Configuration	Hidden	Layers	Test RMSE	Test MAE	Test MAPE (%)
GRU(16, 1L)	16	1	33.6744	23.9373	2386.06
GRU(32, 1L)	32	1	33.5050	23.6583	1761.05
GRU(64, 1L)	64	1	33.6784	23.5038	1358.21
GRU(128, 1L)	128	1	33.6350	23.5872	1540.52
GRU(32, 2L)	**32**	**2**	**33.4780**	**23.4008**	**1450.35**
GRU(64, 2L)	64	2	33.7509	23.6682	1570.88

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.

The best GRU configuration is GRU(32, 2L), a two-layer model with 32 hidden units per layer, achieving a test RMSE of 33.48 mm/week. GRU configurations produce higher RMSE than LSTM overall, contrasting with patterns sometimes observed on larger datasets where GRU efficiency leads to better generalization.

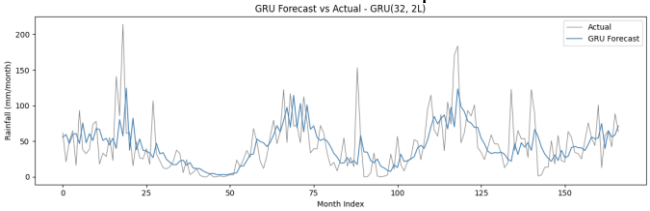
Forecasting Performance of chosen best GRU model is as follows:

TABEL XIX. BEST GRU (32, 2L) FORECASTING PERFORMANCE

Set	RMSE	MAE	MAPE (%)
Train	31.5784	20.5691	303.12
Test	33.4780	23.4008	1450.35

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.

GRU achieves a test RMSE of 33.48 mm/week, ranking fifth overall, behind LSTM, NN, and all non-seasonal statistical models. The train-to-test increase (31.58 to 33.48) is modest and consistent with the other deep learning models. Here is the visualization of the model's performance:



4.12 Overall Model Comparison

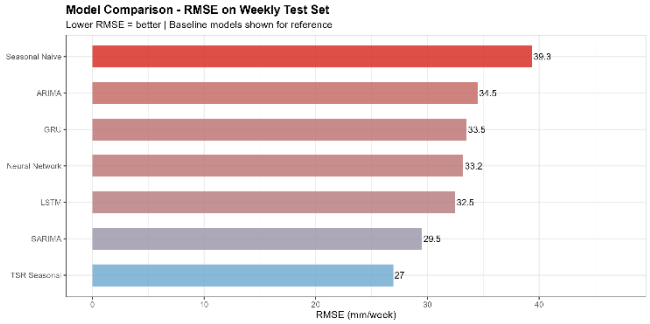
4.12.1 Weekly Test Set Performance

Table XX shows how all seven models performed on the weekly test set covering October 2022 to January 2026, ranked from best to worst based on test RMSE. All models were trained and tested on the exact same 835-observation weekly dataset, which was built using a consistent rolling 7-day block aggregation across both the R and Python implementations.

TABEL XX. FINAL MODEL COMPARISON: WEEKLY TEST SET (ORDERED BY TEST RMSE)

	Model	Train RMSE	Train MAE	Test RMSE	Test MAE	Test MAPE (%)
1	**TSR (Lag1 +Lag2 +Lag3 +Trend+ Dummies)**	**23.805**	**17.826**	**26.951**	**20.378**	**1890.2**
2	SARIMA (2,0,2) (0,1,1)[52]	24.448	19.183	29.488	20.839	908.05
3	LSTM (64, 1L)	32.697	21.126	32.457	23.106	1803.6
4	NN (52, 1HL-32)	28.024	18.210	33.196	22.646	1614.4
5	GRU (32, 2L)	31.578	20.569	33.478	23.400	1450.3
6	ARIMA (3,0,2)	26.072	18.490	34.475	25.451	2802.7
7	Seasonal Naive	36.042	26.966	39.348	30.242	2725.9

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.



TSR retains the best overall performance (test RMSE = 26.95 mm/week), followed by SARIMA (29.49 mm/week). Among deep learning models, LSTM(64, 1L) ranks third (32.46 mm/week), NN(1HL-32) fourth (33.20 mm/week), and GRU(32, 2L) fifth (33.48 mm/week). Across all three deep learning families, the best configurations are the simpler and shallower architectures, reinforcing the training-sample-size constraint with approximately 13 available training annual cycles.

The structured seasonal models maintain a consistent gap of approximately 3.5 to 6.5 mm/week over all deep learning approaches. TSR and SARIMA encode the annual seasonal cycle directly, via 51 week dummy variables and seasonal differencing with a seasonal MA term respectively, while deep learning models must infer this structure implicitly from the 52-step input window. ARIMA ranks sixth, confirming that autoregressive structure without seasonal components is inadequate for the 167-week forecast horizon. Both baselines occupy the two lowest positions, validating all six structured and deep learning approaches.

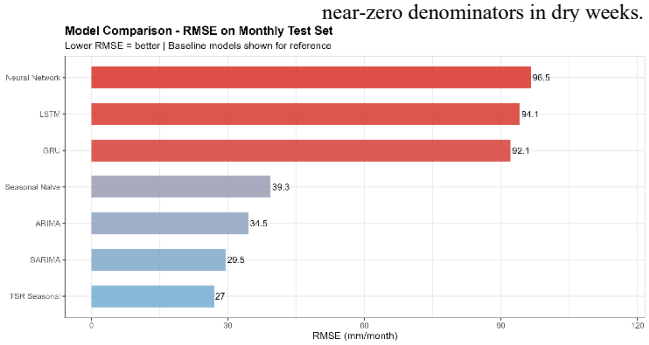
4.13 Monthly Aggregation Comparison

As a supplementary analysis, all seven models were applied to the monthly aggregated rainfall series (193 monthly observations with 154 training and 39 test). Monthly results serve to examine the effect of temporal resolution on relative model performance and are not used for primary model selection.

TABEL XXI. FINAL MODEL COMPARISON: MONTHLY TEST SET (ORDERED BY TEST RMSE)

	Model	Test RMSE	Test MAE	Test MAPE (%)
1	**TSR Seasonal**	**83.6163**	**59.2085**	**223.24**
2	GRU	92.0993	70.9792	244.10
3	SARIMA	93.2849	69.4894	208.37
4	LSTM	94.1221	72.5429	265.08
5	Neural Network	96.5494	69.4857	247.48
6	Seasonal Naive	97.0863	77.2300	302.87
7	ARIMA	108.0335	80.4723	266.99

*MAPE computed on non-zero weeks only. High values reflect near-zero denominators in dry weeks.



Moving from weekly to monthly aggregation, absolute RMSE values roughly double across all models. From the 26–39 mm/week range to the 84–108 mm/month range. This increase is expected given that monthly rainfall totals are approximately 4x larger than weekly totals, making direct RMSE comparison across resolutions misleading. Normalizing by the series mean reveals that TSR's CV-RMSE improves from 62.2% at the weekly level to 41.9% at the monthly level, suggesting that the smoother monthly series is structurally easier to forecast on a relative basis. Deep learning models show a similar pattern: GRU's CV-RMSE drops from 77.2% weekly to 46.2% monthly, a larger proportional improvement than TSR, consistent with the hypothesis that implicit seasonal learning becomes more reliable when high-frequency noise is reduced through aggregation.

4.14 Model Selection

Based on how each model performed on the weekly test set, the Time Series Regression model that includes three autoregressive lags, a linear trend, and 51 weekly seasonal dummy variables, which we refer to as TSR(Lag1+ Lag2+ Lag3+ Trend+ Dummies), came out as the best model overall. It got a test RMSE of 26.95 mm/week and a test MAE of 20.38 mm/week, beating all six other models including the more complex deep learning approaches and the explicitly seasonal SARIMA.

V. CONCLUSION

This study tested a range of time series and machine learning models to forecast weekly rainfall in Jakarta, focusing on how seasonality, model complexity, and statistical validity affect the results. One thing that came out very clearly is that models which directly account for seasonal patterns tend to perform better. Both TSR and SARIMA, which explicitly build in the annual rainfall cycle, outperformed the other approaches at both the weekly and monthly levels, showing just how important it is to capture Jakarta's monsoon pattern in the model.

On the other hand, making a model more complex does not automatically make it better. For the neural network, LSTM, and GRU models, the simpler and shallower configurations actually did better than the deeper and wider ones. This is probably because the dataset only covers 13 annual cycles, which is not a lot, and more complex models tend to overfit when data is limited. This basically tells us that model complexity should match how much data you actually have.

From a statistical standpoint, the SARIMA model we selected is still a solid choice even though one of its parameters, the ma1 coefficient, turned out to be individually non-significant. When we compared models using AIC, the full specification still gave a better overall fit, so we kept it. Also, even though the residuals from the ARIMA-based models are not perfectly normal, this does not hurt their forecasting performance. Both ARIMA and SARIMA passed the Ljung-Box test, which means the residuals are independent enough to meet the key assumptions for time series modeling.

The log(1+x) transformation we applied was backed up by the Box-Cox analysis, which confirmed that variance stabilization was needed. This step helped improve model performance. We also found that MAPE is not a great metric for this dataset because of the dry weeks where rainfall is zero or close to zero, which causes the MAPE formula to break down and produce unreliably large numbers. Because of this, we focused mainly on RMSE and MAE as our primary evaluation metrics since they give more stable and meaningful results.

Overall, the results suggest that simpler models with a clear seasonal structure work best for forecasting Jakarta rainfall. TSR gave the best point forecast performance and is easy to interpret, making it a practical choice for real-world use. If uncertainty estimates are needed, like for flood risk management or reservoir planning, SARIMA is a strong alternative since it can also provide probabilistic forecasts while still being competitive in accuracy. Future studies could look into hybrid models or adding external variables to see if forecasting performance can be improved even further.

ACKNOWLEDGMENT

We would like to thank our lecturer, Syarifah Diana Permai S.Si., M.Si., for all the guidance, support, and encouragement she gave us throughout this research. Her knowledge and dedication really made a big difference in helping us complete this study.

REFERENCES

[1] Badan Penanggulangan Bencana Daerah (BPBD) DKI Jakarta, "Data Banjir DKI Jakarta 2020–2025," 2025. [Online]. Available: https://bpbd.jakarta.go.id

[2] M. R. B. E. Prasetyo, "BNPB: Kerugian bencana banjir di Jabodetabek capai Rp1,69 triliun," *ANTARA News*, Mar. 27, 2025. [Online]. Available: https://www.antaranews.com/berita/4741733/bnpb-kerugian-bencana-banjir-di-jabodetabek-capai-rp169-triliun

[3] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ: Wiley, 2015. [Online]. Available: https://repo.darmajaya.ac.id/4781/1/Time%20Series%20Analysis_%20Forecasting%20and%20Control%20%28%20PDFDrive%20%29.pdf

[4] T. A. Sukri, I. Sugiarto, and A. Rachmansyah, "Rainfall forecasting using LSTM recurrent neural network," *MDPI Water*, vol. 10, no. 11, p. 1655, 2018. [Online]. Available: https://www.mdpi.com/2073-4441/10/11/1655

[5] F. A. Khosravi, "Forecasting monthly discharge using machine learning techniques," *ResearchGate*, 2021. [Online]. Available: https://www.researchgate.net/publication/349530984_Forecasting_Monthly_Discharge_Using_Machine_Learning_Techniques

[6] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," *International Journal of Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0169207021001874

[7] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[8] R. M. Adnan *et al.*, "Estimating reference evapotranspiration using hybrid adaptive fuzzy inferencing coupled with heuristic algorithms," *Computers and Electronics in Agriculture*, vol. 191, p. 106530, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0168169921005585

[9] A. Ridwan *et al.*, "Forecasting of rainfall in Malaysia using time series analysis," *ResearchGate*, 2024. [Online]. Available: https://www.researchgate.net/publication/398105858_Forecasting_of_Rainfall_in_Malaysia_Using_Time_Series_Analysis

[10] Rahmadya Trias Handayanto *et al.*, "Comparison of seasonal ARIMA (SARIMA) and seasonal ARIMAX (SARIMAX) models in forecasting monthly rainfall intensity of Bogor Regency," *ResearchGate*, 2023. [Online]. Available: https://www.researchgate.net/publication/401004625_Comparison_of_seasonal_ARIMA_SARIMA_and_seasonal_ARIMAX_SARIMAX_models_in_forecasting_monthly_rainfall_intensity_of_Bogor_Regency

[11] G. P. Zhang, "Time series forecasting using a hybrid ARIMA and neural network model," *Neurocomputing*, vol. 50, pp. 159–175, 2003. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S0925231201007020

[12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://www.researchgate.net/publication/13853244_Long_Short-Term_Memory

[13] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," *arXiv:1406.1078*, 2014. [Online]. Available: https://arxiv.org/abs/1406.1078

[14] S. Poornima and M. Pushpalatha, "Rainfall prediction using machine learning approaches: A review," *Ecological Informatics*, vol. 77, p. 102244, 2023. [Online]. Available: https://www.researchgate.net/publication/319503839_Machine_Learning_Techniques_For_Rainfall_Prediction_A_Review

APPENDICES

APPENDIX A. R CODE WEEKLY AGGREGATE

A.1 Import Libraries

```
library(tidyverse)
library(lubridate)
library(forecast)
library(tseries)
library(TTR)
library(car)
library(Metrics)
library(moments)
library(ggplot2)
library(knitr)
library(kableExtra)
```

A.2 Load Daily Rainfall Data

```
df_daily <- read.csv("daily_rainfall.csv")

df_daily$Date <- as.Date(df_daily$Date)

head(df_daily)
str(df_daily)
summary(df_daily)
```

A.3 Daily Rainfall Descriptive Statistics

```
desc_daily <- df_daily %>%
  summarise(
    Mean = round(mean(rain, na.rm = TRUE), 3),
    Median = round(median(rain, na.rm = TRUE), 3),
    SD = round(sd(rain, na.rm = TRUE), 3),
    Min = round(min(rain, na.rm = TRUE), 3),
    Max = round(max(rain, na.rm = TRUE), 3),
    Skewness = round(moments::skewness(na.omit(rain)), 3),
    Kurtosis = round(moments::kurtosis(na.omit(rain)), 3)
  )

kable(desc_daily)
```

A.4 Daily Rainfall Distribution

```
ggplot(df_daily, aes(x = rain)) +
  geom_histogram(
    bins = 40,
    fill = "skyblue",
    color = "black"
  ) +
  labs(
    title = "Daily Rainfall Distribution",
    x = "Rainfall (mm)",
    y = "Frequency"
  ) +
  theme_bw()
```

A.5 Weekly Aggregation

```
df_weekly <- df_daily %>%
  mutate(
    WEEK_ID = floor(as.numeric(Date - min(Date))/7) + 1
  ) %>%
  group_by(WEEK_ID) %>%
  summarise(
    week_start = min(Date),
    week_end = max(Date),
    rain_weekly = sum(rain, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(WEEK_ID) %>%
  mutate(
    t = row_number(),
    rain_log = log1p(rain_weekly),
    lag1 = lag(rain_weekly, 1),
    lag2 = lag(rain_weekly, 2),
    lag3 = lag(rain_weekly, 3)
  )

head(df_weekly)
```

A.6 Weekly Descriptive Statistics

```
weekly_stats <- df_weekly %>%
  summarise(
    Mean = round(mean(rain_weekly), 3),
    Median = round(median(rain_weekly), 3),
    SD = round(sd(rain_weekly), 3),
    Min = round(min(rain_weekly), 3),
    Max = round(max(rain_weekly), 3),
    Skewness = round(
      moments::skewness(rain_weekly),
      3
    ),
    Kurtosis = round(
      moments::kurtosis(rain_weekly),
      3
    )
  )

kable(weekly_stats)
```

A.7 Weekly Time Series Object

```
weekly_ts <- ts(
  df_weekly$rain_weekly,
  frequency = 52
)

plot(
  weekly_ts,
  main = "Weekly Rainfall Time Series",
  ylab = "Rainfall (mm/week)"
)
```

A.8 Box-Cox Transformation

```
lambda <- BoxCox.lambda(
  weekly_ts
)

lambda

BoxCox.lambda(weekly_ts)

plot(
  BoxCox(weekly_ts, lambda),
  main = "Box-Cox Transformed Weekly Rainfall"
)
```

A.9 Log Transformation

```
weekly_log <- log1p(
  df_weekly$rain_weekly
)

weekly_log_ts <- ts(
  weekly_log,
  frequency = 52
)

plot(
  weekly_log_ts,
  main = "Log-Transformed Weekly Rainfall"
)
```

A.10 Stationarity Analysis

```
adf.test(
  weekly_log_ts
)

par(mfrow = c(1,2))

Acf(
  weekly_log_ts,
  main = "ACF"
)

Pacf(
  weekly_log_ts,
  main = "PACF"
)
```

A.11 Train-Test Split

```
n <- length(weekly_log_ts)

train_size <- floor(
  0.8 * n
)

train_ts <- weekly_log_ts[
  1:train_size
]

test_ts <- weekly_log_ts[
  (train_size+1):n
]

length(train_ts)
length(test_ts)
```

A.12 Seasonal Naive Model

```
fit_snaive <- snaive(
  train_ts,
  h = length(test_ts)
)

yhat_snaive <- expm1(
  as.numeric(
    fit_snaive$mean
  )
)

actual_test <- expm1(
  as.numeric(test_ts)
)

rmse_snaive <- rmse(
  actual_test,
  yhat_snaive
)

mae_snaive <- mae(
  actual_test,
  yhat_snaive
)
```

A.13 TSR Model

```
tsr_df <- df_weekly %>%
  select(
    rain_weekly,
    t,
    lag1,
    lag2,
    lag3
  ) %>%
  mutate(
    week =
      factor(
        ((t-1) %% 52)+1
      )
  ) %>%
  na.omit()
```

```
tsr_model <- lm(
  rain_weekly ~
    lag1 +
    lag2 +
    lag3 +
    t +
    week,
  data = tsr_df
)

summary(tsr_model)

vif(tsr_model)
```

```
yhat_tsr <- numeric(
  length(actual_test)
)

history <- df_weekly$rain_weekly[
  1:train_size
]

for(i in 1:length(actual_test)){

  lag1 <- tail(history,1)
  lag2 <- tail(history,2)[1]
  lag3 <- tail(history,3)[1]

  t_now <- length(history)+1

  week_now <- factor(
    ((t_now-1) %% 52)+1,
    levels =
      levels(
        tsr_df$week
      )
  )

  pred <- predict(
    tsr_model,
    newdata =
      data.frame(
        lag1=lag1,
        lag2=lag2,
        lag3=lag3,
        t=t_now,
        week=week_now
      )
  )

  yhat_tsr[i] <- pred

  history <- c(
    history,
```

```
    pred
  )
}
```

A.14 ARIMA Model

```
fit_arima <- Arima(
  train_ts,
  order = c(3,0,2)
)

summary(
  fit_arima
)

fc_arima <- forecast(
  fit_arima,
  h = length(test_ts)
)

yhat_arima <- expm1(
  as.numeric(
    fc_arima$mean
  )
)
```

A.15 SARIMA Model

```
fit_sarima <- Arima(
  train_ts,
  order = c(2,0,2),
  seasonal =
    list(
      order =
        c(0,1,1),
      period = 52
    )
)

summary(
  fit_sarima
)

fc_sarima <- forecast(
  fit_sarima,
  h = length(test_ts)
)

yhat_sarima <- expm1(
  as.numeric(
    fc_sarima$mean
  )
)
```

A.16 Model Evaluation

```
results <- data.frame(
  Model = c(
    "Seasonal Naive",
    "TSR",
    "ARIMA",
    "SARIMA"
  ),
  RMSE = c(
    rmse(
      actual_test,
      yhat_snaive
    ),
    rmse(
      actual_test,
      yhat_tsr
    ),
    rmse(
      actual_test,
      yhat_arima
    ),
    rmse(
      actual_test,
      yhat_sarima
    )
  ),
  MAE = c(
    mae(
      actual_test,
      yhat_snaive
    ),
    mae(
      actual_test,
      yhat_tsr
    ),
    mae(
      actual_test,
      yhat_arima
    ),
    mae(
      actual_test,
      yhat_sarima
    )
  )
)

kable(results)
```

A.17 Forecast vs Actual Plots

```
plot_forecast <- function(
  actual,
  forecast,
  model_name
){

  df <- data.frame(
    Index =
      seq_along(actual),
    Actual = actual,
    Forecast = forecast
  )

  ggplot(
    df,
    aes(Index)
  ) +
    geom_line(
      aes(
        y = Actual,
        color = "Actual"
      )
    ) +
    geom_line(
      aes(
        y = Forecast,
        color = model_name
      )
    ) +
    labs(
      title =
        paste(
          model_name,
          "Forecast vs Actual"
        ),
      x = "Week Index",
      y = "Rainfall (mm/week)"
    ) +
    scale_color_manual(
      values = c(
        "Actual" = "black",
        model_name = "#2C7FB8"
      )
    ) +
    theme_bw()
}
```

Generate Plots

```
p_snaive <- plot_forecast(
  actual_test,
  yhat_snaive,
  "Seasonal Naive"
)

p_tsr <- plot_forecast(
  actual_test,
  yhat_tsr,
  "TSR"
```

```
)

p_arima <- plot_forecast(
  actual_test,
  yhat_arima,
  "ARIMA"
)

p_sarima <- plot_forecast(
  actual_test,
  yhat_sarima,
  "SARIMA"
)

dir.create(
  "plots",
  showWarnings = FALSE
)

ggsave(
  "plots/TSR.png",
  p_tsr,
  width = 10,
  height = 5,
  dpi = 300
)

ggsave(
  "plots/ARIMA.png",
  p_arima,
  width = 10,
  height = 5,
  dpi = 300
)

ggsave(
  "plots/SARIMA.png",
  p_sarima,
  width = 10,
  height = 5,
  dpi = 300
)

ggsave(
  "plots/SNAIVE.png",
  p_snaive,
  width = 10,
  height = 5,
  dpi = 300
)
```

APPENDIX B. PYTHON CODE WEEKLY AGGREGATE

B.1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error
)

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import (
    Dense,
    LSTM,
    GRU,
    SimpleRNN,
    Dropout
)

from tensorflow.keras.callbacks import (
    EarlyStopping
)

from math import sqrt
```

B.2 Load Weekly Rainfall Data

```
df = pd.read_csv(
    "weekly_rainfall.csv"
)

df.head()

print(df.info())

print(df.describe())
```

B.3 Visualize Weekly Rainfall

```
plt.figure(figsize=(12,5))

plt.plot(
    df["rain_weekly"]
)

plt.title(
    "Weekly Rainfall Time Series"
)

plt.xlabel(
    "Week Index"
)

plt.ylabel(
    "Rainfall (mm/week)"
)

plt.grid(True)

plt.show()
```

B.4 Log Transformation

```
df["rain_log"] = np.log1p(
    df["rain_weekly"]
)

plt.figure(figsize=(12,5))

plt.plot(
    df["rain_log"]
)

plt.title(
    "Log-Transformed Weekly Rainfall"
)

plt.xlabel(
    "Week Index"
)

plt.ylabel(
    "Log Rainfall"
)

plt.grid(True)

plt.show()
```

B.5 Data Scaling

```
scaler = MinMaxScaler()

scaled_data = scaler.fit_transform(
    df[["rain_log"]]
)

scaled_data[:5]
```

B.6 Create Sequence Function

```
def create_sequences(
    data,
    seq_length
):

    X = []

    y = []
```

```
    for i in range(
        len(data) - seq_length
    ):

        X.append(
            data[
                i:i+seq_length
            ]
        )

        y.append(
            data[
                i+seq_length
            ]
        )

    return (
        np.array(X),
        np.array(y)
    )
```

B.7 Define Sequence Length

```
SEQ_LENGTH = 12

X, y = create_sequences(
    scaled_data,
    SEQ_LENGTH
)

print(X.shape)

print(y.shape)
```

B.8 Train-Test Split

```
train_size = int(
    len(X) * 0.8
)

X_train = X[
    :train_size
]

X_test = X[
    train_size:
]

y_train = y[
    :train_size
]

y_test = y[
    train_size:
]

print(X_train.shape)

print(X_ test.shape)
```

B.9 Neural Network (Simple RNN) Model

```
rnn_model = Sequential()

rnn_model.add(
    SimpleRNN(
        64,
        activation="tanh",
        input_shape=(
            X_train.shape[1],
            X_train.shape[2]
        )
    )
)

rnn_model.add(
    Dropout(0.2)
)

rnn_model.add(
    Dense(1)
)

rnn_model.compile(
    optimizer="adam",
    loss="mse"
)

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=20,
    restore_best_weights=True
)

history_rnn = rnn_model.fit(
    X_train,
    y_train,
    epochs=200,
    batch_size=16,
    validation_split=0.1,
    callbacks=[early_stop],
    verbose=1
)
```

B.10 LSTM Model

```
lstm_model = Sequential()

lstm_model.add(
    LSTM(
        64,
        activation="tanh",
        input_shape=(
            X_train.shape[1],
            X_train.shape[2]
        )
    )
)

lstm_model.add(
    Dropout(0.2)
)

lstm_model.add(
    Dense(1)
)

lstm_model.compile(
    optimizer="adam",
    loss="mse"
)

history_lstm = lstm_model.fit(
    X_train,
    y_train,
    epochs=200,
    batch_size=16,
    validation_split=0.1,
    callbacks=[early_stop],
    verbose=1
)
```

B.11 GRU Model

```
gru_model = Sequential()

gru_model.add(
    GRU(
        64,
        activation="tanh",
        input_shape=(
            X_train.shape[1],
            X_train.shape[2]
        )
    )
)

gru_model.add(
    Dropout(0.2)
)

gru_model.add(
    Dense(1)
```

```
)

gru_model.compile(
    optimizer="adam",
    loss="mse"
)

history_gru = gru_model.fit(
    X_train,
    y_train,
    epochs=200,
    batch_size=16,
    validation_split=0.1,
    callbacks=[early_stop],
    verbose=1
)
```

B.12 Generate Predictions

```
pred_rnn = rnn_model.predict(
    X_test
)

pred_lstm = lstm_model.predict(
    X_test
)

pred_gru = gru_model.predict(
    X_test
)
```

B.13 Inverse Scaling

```
y_test_inv = scaler.inverse_transform(
    y_test
)

pred_rnn_inv = scaler.inverse_transform(
    pred_rnn
)

pred_lstm_inv = scaler.inverse_transform(
    pred_lstm
)

pred_gru_inv = scaler.inverse_transform(
    pred_gru
)

actual = np.expm1(
    y_test_inv.flatten()
)

rnn_forecast = np.expm1(
    pred_rnn_inv.flatten()
)

lstm_forecast = np.expm1(
    pred_lstm_inv.flatten()
)

gru_forecast = np.expm1(
    pred_gru_inv.flatten()
)
```

B.14 Model Evaluation

```
results = pd.DataFrame({

    "Model": [
        "Simple RNN",
        "LSTM",
        "GRU"
    ],

    "RMSE": [

        sqrt(
            mean_squared_error(
                actual,
                rnn_forecast
            )
        ),

        sqrt(
            mean_squared_error(
                actual,
                lstm_forecast
            )
        ),

        sqrt(
            mean_squared_error(
                actual,
                gru_forecast
            )
        )
    ],

    "MAE": [

        mean_absolute_error(
            actual,
            rnn_forecast
        ),

        mean_absolute_error(
            actual,
            lstm_forecast
        ),

        mean_absolute_error(
            actual,
            gru_forecast
        )
    ]
})

print(results)
```

B.15 Forecast vs Actual Plot Function

```
def plot_forecast(
    actual,
    forecast,
    model_name
):

    plt.figure(
        figsize=(12,5)
    )

    plt.plot(
        actual,
        label="Actual"
    )

    plt.plot(
        forecast,
        label=model_name
    )

    plt.title(
        f"{model_name} Forecast vs Actual"
    )

    plt.xlabel(
        "Week Index"
    )

    plt.ylabel(
        "Rainfall (mm/week)"
    )

    plt.legend()

    plt.grid(True)

    plt.show()
```

B.16 Plot Forecast Results

```
plot_forecast(
    actual,
```

```
    rnn_forecast,
    "Simple RNN"
)

plot_forecast(
    actual,
    lstm_forecast,
    "LSTM"
)

plot_forecast(
    actual,
    gru_forecast,
    "GRU"
)
```

B.17 Training Loss Visualization

```
plt.figure(figsize=(10,5))

plt.plot(
    history_rnn.history["loss"],
    label="RNN Train Loss"
)

plt.plot(
    history_rnn.history["val_loss"],
    label="RNN Validation Loss"
)

plt.title(
    "Simple RNN Training Loss"
)

plt.xlabel(
    "Epoch"
)

plt.ylabel(
    "Loss"
)

plt.legend()

plt.grid(True)

plt.show()
```

```
plt.figure(figsize=(10,5))

plt.plot(
    history_lstm.history["loss"],
    label="LSTM Train Loss"
)

plt.plot(
    history_lstm.history["val_loss"],
    label="LSTM Validation Loss"
)

plt.title(
    "LSTM Training Loss"
)

plt.xlabel(
    "Epoch"
)

plt.ylabel(
    "Loss"
)

plt.legend()

plt.grid(True)

plt.show()
```

```
plt.figure(figsize=(10,5))

plt.plot(
    history_gru.history["loss"],
    label="GRU Train Loss"
)

plt.plot(
    history_gru.history["val_loss"],
    label="GRU Validation Loss"
)

plt.title(
    "GRU Training Loss"
)

plt.xlabel(
    "Epoch"
)

plt.ylabel(
    "Loss"
)

plt.legend()

plt.grid(True)

plt.show()
```

```
import os

os.makedirs(
    "plots",
    exist_ok=True
)

plt.figure(figsize=(12,5))

plt.plot(actual)

plt.plot(rnn_forecast)

plt.title(
    "Simple RNN Forecast vs Actual"
)

plt.savefig(
    "plots/RNN.png",
    dpi=300,
    bbox_inches="tight"
)

plt.close()
```

```
plt.figure(figsize=(12,5))

plt.plot(actual)

plt.plot(lstm_forecast)

plt.title(
    "LSTM Forecast vs Actual"
)

plt.savefig(
    "plots/LSTM.png",
    dpi=300,
    bbox_inches="tight"
)

plt.close()
```

```
plt.figure(figsize=(12,5))

plt.plot(actual)

plt.plot(gru_forecast)

plt.title(
    "GRU Forecast vs Actual"
)
```

```
plt.savefig(
    "plots/GRU.png",
    dpi=300,
    bbox_inches="tight"
)

plt.close()
```

APPENDIX C. R CODE MONTHLY AGGREGATE

C.1 Import Libraries

```
library(tidyverse)
library(lubridate)
library(forecast)
library(tseries)
library(TTR)
library(car)
library(Metrics)
library(moments)
library(ggplot2)
library(knitr)
library(kableExtra)
```

C.2 Load Daily Rainfall Data

```
df_daily <- read.csv(
  "daily_rainfall.csv"
)

df_daily$Date <- as.Date(
  df_daily$Date
)

head(df_daily)

str(df_daily)

summary(df_daily)
```

C.3 Daily Rainfall Descriptive Statistics

```
desc_daily <- df_daily %>%
  summarise(
    Mean = round(
      mean(rain, na.rm = TRUE),
      3
    ),

    Median = round(
      median(rain, na.rm = TRUE),
      3
    ),

    SD = round(
      sd(rain, na.rm = TRUE),
      3
    ),

    Min = round(
      min(rain, na.rm = TRUE),
      3
    ),

    Max = round(
      max(rain, na.rm = TRUE),
      3
    ),

    Skewness = round(
      moments::skewness(
        na.omit(rain)
      ),
      3
    ),

    Kurtosis = round(
      moments::kurtosis(
        na.omit(rain)
      ),
      3
    )
  )

kable(desc_daily)
```

C.4 Daily Rainfall Distribution

```
ggplot(
  df_daily,
  aes(x = rain)
) +

  geom_histogram(
    bins = 40,
    fill = "skyblue",
    color = "black"
  ) +

  labs(
    title =
      "Daily Rainfall Distribution",

    x = "Rainfall (mm)",

    y = "Frequency"
  ) +

  theme_bw()
```

C.5 Monthly Aggregation

```
df_monthly <- df_daily %>%

  mutate(
    YearMonth =
      floor_date(
        Date,
        unit = "month"
      )
  ) %>%

  group_by(YearMonth) %>%

  summarise(
    rain_monthly =
      sum(
        rain,
        na.rm = TRUE
      ),

    .groups = "drop"
  ) %>%

  arrange(YearMonth) %>%

  mutate(
    t = row_number(),

    rain_log =
      log1p(rain_monthly),

    lag1 = lag(
      rain_monthly,
      1
    ),

    lag2 = lag(
      rain_monthly,
      2
    ),

    lag3 = lag(
      rain_monthly,
      3
    )
  )

head(df_monthly)
```

C.6 Monthly Descriptive Statistics

```
monthly_stats <- df_monthly %>%

  summarise(
```

```
    Mean = round(
      mean(rain_monthly),
      3
    ),

    Median = round(
      median(rain_monthly),
      3
    ),

    SD = round(
      sd(rain_monthly),
      3
    ),

    Min = round(
      min(rain_monthly),
      3
    ),

    Max = round(
      max(rain_monthly),
      3
    ),

    Skewness = round(
      moments::skewness(
        rain_monthly
      ),
      3
    ),

    Kurtosis = round(
      moments::kurtosis(
        rain_monthly
      ),
      3
    )
  )

kable(monthly_stats)
```

C.7 Monthly Time Series Object

```
monthly_ts <- ts(
  df_monthly$rain_monthly,
  frequency = 12
)

plot(
  monthly_ts,

  main =
    "Monthly Rainfall Time Series",

  ylab =
    "Rainfall (mm/month)"
)
```

C.8 Box-Cox Transformation

```
lambda <- BoxCox.lambda(
  monthly_ts
)

lambda

BoxCox.lambda(
  monthly_ts
)

plot(
  BoxCox(
    monthly_ts,
    lambda
  ),

  main =
    "Box-Cox Transformed Monthly Rainfall"
)
```

C.9 Log Transformation

```
monthly_log <- log1p(
  df_monthly$rain_monthly
)

monthly_log_ts <- ts(
  monthly_log,
  frequency = 12
)

plot(
  monthly_log_ts,

  main =
    "Log-Transformed Monthly Rainfall"
)
```

C.10 Stationarity Analysis

```
adf.test(
  monthly_log_ts
)

par(
  mfrow = c(1,2)
)

Acf(
  monthly_log_ts,
  main = "ACF"
)

Pacf(
  monthly_log_ts,
  main = "PACF"
)
```

C.11 Train-Test Split

```
n <- length(
  monthly_log_ts
)

train_size <- floor(
  0.8 * n
)

train_ts <- monthly_log_ts[
  1:train_size
]

test_ts <- monthly_log_ts[
  (train_size+1):n
]

length(train_ts)

length(test_ts)
```

C.12 Seasonal Naive Model

```
fit_snaive <- snaive(
  train_ts,
  h = length(test_ts)
)

yhat_snaive <- expm1(
  as.numeric(
    fit_snaive$mean
  )
)

actual_test <- expm1(
  as.numeric(test_ts)
)

rmse_snaive <- rmse(
  actual_test,
  yhat_snaive
)

mae_snaive <- mae(
  actual_test,
  yhat_snaive
```

```
)
```

C.13 TSR Model

```
tsr_df <- df_monthly %>%

  select(
    rain_monthly,
    t,
    lag1,
    lag2,
    lag3
  ) %>%

  mutate(
    month =
      factor(
        ((t-1) %% 12)+1
      )
  ) %>%

  na.omit()

tsr_model <- lm(
  rain_monthly ~

    lag1 +

    lag2 +

    lag3 +

    t +

    month,

  data = tsr_df
)

summary(tsr_model)

vif(tsr_model)
```

```
yhat_tsr <- numeric(
  length(actual_test)
)

history <- df_monthly$rain_monthly[
  1:train_size
]

for(i in 1:length(actual_test)){

  lag1 <- tail(
    history,
    1
  )

  lag2 <- tail(
    history,
    2
  )[1]

  lag3 <- tail(
    history,
    3
  )[1]

  t_now <- length(history)+1

  month_now <- factor(
    ((t_now-1) %% 12)+1,

    levels =
      levels(
        tsr_df$month
      )
  )

  pred <- predict(
    tsr_model,

    newdata =
      data.frame(
        lag1=lag1,
        lag2=lag2,
        lag3=lag3,
        t=t_now,
        month=month_now
      )
  )

  yhat_tsr[i] <- pred

  history <- c(
    history,
    pred
  )
}
```

C.14 ARIMA Model

```
fit_arima <- Arima(
  train_ts,
  order = c(2,0,2)
)

summary(
  fit_arima
)

fc_arima <- forecast(
  fit_arima,
  h = length(test_ts)
)

yhat_arima <- expm1(
  as.numeric(
    fc_arima$mean
  )
)
```

C.15 SARIMA Model

```
fit_sarima <- Arima(

  train_ts,

  order = c(1,0,1),

  seasonal =
    list(
      order =
        c(0,1,1),

      period = 12
    )
)

summary(
  fit_sarima
)

fc_sarima <- forecast(
  fit_sarima,
  h = length(test_ts)
)

yhat_sarima <- expm1(
  as.numeric(
    fc_sarima$mean
  )
)
```

C.16 Model Evaluation

```
results <- data.frame(

  Model = c(
    "Seasonal Naive",
    "TSR",
    "ARIMA",
    "SARIMA"
```

```
  ),

  RMSE = c(

    rmse(
      actual_test,
      yhat_snaive
    ),

    rmse(
      actual_test,
      yhat_tsr
    ),

    rmse(
      actual_test,
      yhat_arima
    ),

    rmse(
      actual_test,
      yhat_sarima
    )
  ),

  MAE = c(

    mae(
      actual_test,
      yhat_snaive
    ),

    mae(
      actual_test,
      yhat_tsr
    ),

    mae(
      actual_test,
      yhat_arima
    ),

    mae(
      actual_test,
      yhat_sarima
    )
  )
)

kable(results)
```

C.17 Forecast vs Actual Plot Function

```
plot_forecast <- function(
  actual,
  forecast,
  model_name
){

  df <- data.frame(

    Index =
      seq_along(actual),

    Actual = actual,

    Forecast = forecast
  )

  ggplot(
    df,
    aes(Index)
  ) +

    geom_line(
      aes(
        y = Actual,
        color = "Actual"
      )
    ) +

    geom_line(
      aes(
        y = Forecast,
        color = model_name
      )
    ) +

    labs(

      title =
        paste(
          model_name,
          "Forecast vs Actual"
        ),

      x = "Month Index",

      y =
        "Rainfall (mm/month)"
    ) +

    scale_color_manual(
      values = c(
        "Actual" = "black",
        model_name = "#2C7FB8"
      )
    ) +

    theme_bw()
}
```

C.18 Generate Plots

```
p_snaive <- plot_forecast(
  actual_test,
  yhat_snaive,
  "Seasonal Naive"
)

p_tsr <- plot_forecast(
  actual_test,
  yhat_tsr,
  "TSR"
)

p_arima <- plot_forecast(
  actual_test,
  yhat_arima,
  "ARIMA"
)

p_sarima <- plot_forecast(
  actual_test,
  yhat_sarima,
  "SARIMA"
)
```

```
dir.create(
  "plots",
  showWarnings = FALSE
)

ggsave(
  "plots/TSR_MONTHLY.png",

  p_tsr,

  width = 10,

  height = 5,

  dpi = 300
)

ggsave(
  "plots/ARIMA_MONTHLY.png",

  p_arima,
```

```
  width = 10,

  height = 5,

  dpi = 300
)

ggsave(
  "plots/SARIMA_MONTHLY.png",

  p_sarima,

  width = 10,

  height = 5,

  dpi = 300
)

ggsave(
  "plots/SNAIVE_MONTHLY.png",

  p_snaive,

  width = 10,

  height = 5,

  dpi = 300
)
```

APPENDIX D. PYTHON CODE MONTHLY AGGREGATE

D.1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import (
    MinMaxScaler
)

from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error
)

from tensorflow.keras.models import (
    Sequential
)

from tensorflow.keras.layers import (
    Dense,
    LSTM,
    GRU,
    SimpleRNN,
    Dropout
)

from tensorflow.keras.callbacks import (
    EarlyStopping
)

from math import sqrt
```

D.2 Load Monthly Rainfall Data

```
df = pd.read_csv(
    "monthly_rainfall.csv"
)

df.head()

print(df.info())

print(df.describe())
```

D.3 Visualize Monthly Rainfall

```
plt.figure(figsize=(12,5))

plt.plot(
    df["rain_monthly"]
)

plt.title(
    "Monthly Rainfall Time Series"
)

plt.xlabel(
    "Month Index"
)

plt.ylabel(
    "Rainfall (mm/month)"
)

plt.grid(True)

plt.show()
```

D.4 Log Transformation

```
df["rain_log"] = np.log1p(
    df["rain_monthly"]
)

plt.figure(figsize=(12,5))

plt.plot(
    df["rain_log"]
)

plt.title(
    "Log-Transformed Monthly Rainfall"
)

plt.xlabel(
    "Month Index"
)

plt.ylabel(
    "Log Rainfall"
)

plt.grid(True)

plt.show()
```

D.5 Data Scaling

```
scaler = MinMaxScaler()

scaled_data = scaler.fit_transform(
    df[["rain_log"]]
)

scaled_data[:5]
```

D.6 Create Sequence Function

```
def create_sequences(
    data,
    seq_length
):

    X = []

    y = []

    for i in range(
        len(data) - seq_length
    ):

        X.append(
            data[
                i:i+seq_length
            ]
        )

        y.append(
            data[
                i+seq_length
```

```
            ]
        )

    return (
        np.array(X),
        np.array(y)
    )
```

D.7 Define Sequence Length

```
SEQ_LENGTH = 6

X, y = create_sequences(
    scaled_data,
    SEQ_LENGTH
)

print(X.shape)

print(y.shape)
```

D.8 Train-Test Split

```
train_size = int(
    len(X) * 0.8
)

X_train = X[
    :train_size
]

X_test = X[
    train_size:
]

y_train = y[
    :train_size
]

y_test = y[
    train_size:
]

print(X_train.shape)

print(X_ test.shape)
```

D.9 Neural Network (Simple RNN) Model

```
rnn_model = Sequential()

rnn_model.add(
    SimpleRNN(
        64,
        activation="tanh",
        input_shape=(
            X_train.shape[1],
            X_train.shape[2]
        )
    )
)

rnn_model.add(
    Dropout(0.2)
)

rnn_model.add(
    Dense(1)
)

rnn_model.compile(
    optimizer="adam",
    loss="mse"
)

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=20,
    restore_best_weights=True
)

history_rnn = rnn_model.fit(
    X_train,
    y_train,
    epochs=200,
    batch_size=8,
    validation_split=0.1,
    callbacks=[early_stop],
    verbose=1
)
```

D.10 LSTM Model

```
lstm_model = Sequential()

lstm_model.add(
    LSTM(
        64,
        activation="tanh",
        input_shape=(
            X_train.shape[1],
            X_train.shape[2]
        )
    )
)

lstm_model.add(
    Dropout(0.2)
)

lstm_model.add(
    Dense(1)
)

lstm_model.compile(
    optimizer="adam",
    loss="mse"
)

history_lstm = lstm_model.fit(
    X_train,
    y_train,
    epochs=200,
    batch_size=8,
    validation_split=0.1,
    callbacks=[early_stop],
    verbose=1
)
```

D.11 GRU Model

```
gru_model = Sequential()

gru_model.add(
    GRU(
        64,
        activation="tanh",
        input_shape=(
            X_train.shape[1],
            X_train.shape[2]
        )
    )
)

gru_model.add(
    Dropout(0.2)
)

gru_model.add(
    Dense(1)
)

gru_model.compile(
    optimizer="adam",
    loss="mse"
)

history_gru = gru_model.fit(
    X_train,
    y_train,
    epochs=200,
    batch_size=8,
    validation_split=0.1,
    callbacks=[early_stop],
```

```
    verbose=1
)
```

D.12 Generate Predictions

```
pred_rnn = rnn_model.predict(
    X_test
)

pred_lstm = lstm_model.predict(
    X_test
)

pred_gru = gru_model.predict(
    X_test
)
```

D.13 Inverse Scaling

```
y_test_inv = scaler.inverse_transform(
    y_test
)

pred_rnn_inv = scaler.inverse_transform(
    pred_rnn
)

pred_lstm_inv = scaler.inverse_transform(
    pred_lstm
)

pred_gru_inv = scaler.inverse_transform(
    pred_gru
)

actual = np.expm1(
    y_test_inv.flatten()
)

rnn_forecast = np.expm1(
    pred_rnn_inv.flatten()
)

lstm_forecast = np.expm1(
    pred_lstm_inv.flatten()
)

gru_forecast = np.expm1(
    pred_gru_inv.flatten()
)
```

D.14 Model Evaluation

```
results = pd.DataFrame({

    "Model": [
        "Simple RNN",
        "LSTM",
        "GRU"
    ],

    "RMSE": [

        sqrt(
            mean_squared_error(
                actual,
                rnn_forecast
            )
        ),

        sqrt(
            mean_squared_error(
                actual,
                lstm_forecast
            )
        ),

        sqrt(
            mean_squared_error(
                actual,
                gru_forecast
            )
        )
    ],

    "MAE": [

        mean_absolute_error(
            actual,
            rnn_forecast
        ),

        mean_absolute_error(
            actual,
            lstm_forecast
        ),

        mean_absolute_error(
            actual,
            gru_forecast
        )
    ]
})

print(results)
```

D.15 Forecast vs Actual Plot Function

```
def plot_forecast(
    actual,
    forecast,
    model_name
):

    plt.figure(
        figsize=(12,5)
    )

    plt.plot(
        actual,
        label="Actual"
    )

    plt.plot(
        forecast,
        label=model_name
    )

    plt.title(
        f"{model_name} Forecast vs Actual"
    )

    plt.xlabel(
        "Month Index"
    )

    plt.ylabel(
        "Rainfall (mm/month)"
    )

    plt.legend()

    plt.grid(True)

    plt.show()
```

D.16 Plot Forecast Results

```
plot_forecast(
    actual,
    rnn_forecast,
    "Simple RNN"
)

plot_forecast(
    actual,
    lstm_forecast,
    "LSTM"
)

plot_forecast(
    actual,
    gru_forecast,
    "GRU"
```

```
)
```

D.17 Training Loss Visualization

```
plt.figure(figsize=(10,5))

plt.plot(
    history_rnn.history["loss"],
    label="RNN Train Loss"
)

plt.plot(
    history_rnn.history["val_loss"],
    label="RNN Validation Loss"
)

plt.title(
    "Simple RNN Training Loss"
)

plt.xlabel(
    "Epoch"
)

plt.ylabel(
    "Loss"
)

plt.legend()

plt.grid(True)

plt.show()
```

```
plt.figure(figsize=(10,5))

plt.plot(
    history_lstm.history["loss"],
    label="LSTM Train Loss"
)

plt.plot(
    history_lstm.history["val_loss"],
    label="LSTM Validation Loss"
)

plt.title(
    "LSTM Training Loss"
)

plt.xlabel(
    "Epoch"
)

plt.ylabel(
    "Loss"
)

plt.legend()

plt.grid(True)

plt.show()
```

```
plt.figure(figsize=(10,5))

plt.plot(
    history_gru.history["loss"],
    label="GRU Train Loss"
)

plt.plot(
    history_gru.history["val_loss"],
    label="GRU Validation Loss"
)

plt.title(
    "GRU Training Loss"
)

plt.xlabel(
    "Epoch"
)

plt.ylabel(
    "Loss"
)

plt.legend()

plt.grid(True)

plt.show()
```

```
import os

os.makedirs(
    "plots",
    exist_ok=True
)

plt.figure(figsize=(12,5))

plt.plot(actual)

plt.plot(rnn_forecast)

plt.title(
    "Simple RNN Forecast vs Actual"
)

plt.savefig(
    "plots/RNN_MONTHLY.png",
    dpi=300,
    bbox_inches="tight"
)

plt.close()
```

```
plt.figure(figsize=(12,5))

plt.plot(actual)

plt.plot(lstm_forecast)

plt.title(
    "LSTM Forecast vs Actual"
)

plt.savefig(
    "plots/LSTM_MONTHLY.png",
    dpi=300,
    bbox_inches="tight"
)

plt.close()
```

```
plt.figure(figsize=(12,5))

plt.plot(actual)

plt.plot(gru_forecast)

plt.title(
    "GRU Forecast vs Actual"
)

plt.savefig(
    "plots/GRU_MONTHLY.png",
    dpi=300,
    bbox_inches="tight"
)

plt.close()
```